

**ĐẠI HỌC TRÀ VINH**  
**TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ**



**ĐỒ ÁN TỐT NGHIỆP**

**XÂY DỰNG ỨNG DỤNG**  
**NHẬN DIỆN BỘ CẢNH CỨNG**

Giảng viên hướng dẫn: **TS. NGUYỄN NHÚT LAM**

Sinh viên thực hiện: **NGUYỄN HUỲNH YẾN NHƯ**

Mã số sinh viên: **110122018**

Lớp: **DA22TTA**

Khóa: **2022**

**Vĩnh Long, tháng 6 năm 2026**

**ĐẠI HỌC TRÀ VINH**  
**TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ**



**ĐỒ ÁN TỐT NGHIỆP**

**XÂY DỰNG ỨNG DỤNG**  
**NHẬN DIỆN BỘ CẢNH CỨNG**

Giảng viên hướng dẫn: **TS. NGUYỄN NHÚT LAM**

Sinh viên thực hiện: **NGUYỄN HUỲNH YẾN NHƯ**

Mã số sinh viên: **110122018**

Lớp: **DA22TTA**

Khóa: **2022**

**Vĩnh Long, tháng 6 năm 2026**

## **LỜI CAM ĐOAN**

Em xin cam đoan rằng công trình nghiên cứu này là sản phẩm của chính bản thân, được thực hiện trên tinh thần học thuật trung thực. Các tài liệu tham khảo sử dụng trong khoá luận đều đã được trích dẫn rõ ràng, đầy đủ theo các quy định về đạo đức nghiên cứu.

Em hoàn toàn chịu trách nhiệm về tính chính xác, hợp pháp của toàn bộ nội dung khoá luận.

*Vĩnh Long, ngày 29 tháng 6 năm 2026*

**Sinh viên thực hiện**

**Nguyễn Huỳnh Yến Như**

## LỜI CẢM ƠN

Để hoàn thành khóa luận này, em xin gửi lời cảm ơn chân thành đến Đại học Trà Vinh và Trường Kỹ thuật và Công nghệ đã tạo điều kiện về môi trường học tập cũng như những kiến thức quý báu trong suốt thời gian em theo học tại trường

Em xin bày tỏ lòng biết ơn sâu sắc đến quý thầy, cô đặc biệt là các thầy cô Khoa Công nghệ thông tin, những người đã tận tình giảng dạy và truyền đạt cho em không chỉ kiến thức chuyên môn mà còn cả tinh thần học hỏi, nghiên cứu trong suốt quá trình học tập.

Đặc biệt, em xin gửi lời cảm ơn chân thành nhất đến thầy Nguyễn Nhứt Lam người đã trực tiếp hướng dẫn em thực hiện đề tài. Cảm ơn thầy đã luôn tận tâm chỉ bảo, định hướng và góp ý để em kịp thời khắc phục những thiếu sót và hoàn thiện đề tài.

Em cũng xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn quan tâm, động viên và giúp đỡ em trong suốt thời gian học tập cũng như khi thực hiện đề tài.

Mặc dù đã cố gắng hết sức, nhưng do kiến thức và kinh nghiệm còn hạn chế nên đề tài khó tránh khỏi những thiếu sót. Em rất mong tiếp tục nhận được sự góp ý của quý thầy cô để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

Kính chúc quý thầy cô luôn dồi dào sức khỏe và thành công trong sự nghiệp.

## **NHẬN XÉT**

**(Của giảng viên hướng dẫn trong đề án của sinh viên)**

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

**Giảng viên hướng dẫn**

*(Ký & ghi rõ họ tên)*

**Nguyễn Nhứt Lam**

**UBND TỈNH VĨNH LONG**

**ĐẠI HỌC TRÀ VINH**

**CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM**

**Độc lập – Tự do – Hạnh phúc**

## **BẢN NHẬN XÉT ĐỒ ÁN, KHÓA LUẬN TỐT NGHIỆP**

Họ và tên sinh viên: Nguyễn Huỳnh Yên Như

MSSV: 110122018

Ngành: Công nghệ thông tin

Khóa: 2022

Tên đề tài: Xây dựng ứng dụng nhận diện bộ cánh cứng

Họ và tên Giáo viên hướng dẫn: TS. Nguyễn Nhứt Lam

Chức danh: Giảng viên

Học vị: Tiến Sĩ

### **NHẬN XÉT**

#### 1. Nội dung đề tài:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

#### 2. Ưu điểm:

.....

.....

.....

.....

#### 3. Nhược điểm:

.....

.....

.....

.....

.....

.....

.....

4. Điểm mới đề tài:

.....

.....

.....

.....

.....

.....

.....

.....

.....

5. Giá trị thực trên đề tài:

.....

.....

.....

.....

.....

.....

.....

.....

7. Đề nghị sửa chữa bổ sung:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

8. Đánh giá:

.....

.....

.....

.....

.....

*Vĩnh Long, ngày    tháng    năm 2026*

Giảng viên hướng dẫn

*(Ký & ghi rõ họ tên)*

**Nguyễn Nhứt Lam**



## NHẬN XÉT

(Của giảng viên chấm trong đề án, khóa luận của sinh viên)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

**Giảng viên chấm**

*(Ký & ghi rõ họ tên)*

## **BẢN NHẬN XÉT ĐỒ ÁN, KHÓA LUẬN TỐT NGHIỆP**

*(Của cán bộ chấm đồ án, khóa luận)*

Họ và tên người nhận xét: .....

Chức danh: ..... Học vị: .....

Chuyên ngành: .....

Cơ quan công tác: .....

Tên sinh viên: .....

Tên đề tài đồ án, khóa luận tốt nghiệp: .....

.....

.....

### **I. Ý KIẾN NHẬN XÉT**

#### **1. Nội dung:**

.....

.....

.....

.....

.....

.....

.....

.....

.....

#### **2. Điểm mới các kết quả của đồ án, khóa luận:**

.....

.....

.....

3. Ứng dụng thực tế:

(Các câu hỏi của giáo viên phản biện)

[illegible]

### III. KẾT LUẬN

(Ghi rõ đồng ý hay không đồng ý cho bảo vệ đồ án khóa luận tốt nghiệp)

.....

.....

.....

.....

.....

.....

*Vĩnh Long, ngày ..... tháng ..... năm 2026*

Người nhận xét

*(Ký & ghi rõ họ tên)*

# MỤC LỤC

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>LỜI CAM ĐOAN.....</b>                                     | <b>i</b>  |
| <b>LỜI CẢM ƠN .....</b>                                      | <b>ii</b> |
| <b>Chương 1: MỞ ĐẦU .....</b>                                | <b>1</b>  |
| <b>1.1 Lý do chọn đề tài.....</b>                            | <b>1</b>  |
| <b>1.2 Mục tiêu đề tài.....</b>                              | <b>2</b>  |
| <b>1.3 Phạm vi nghiên cứu .....</b>                          | <b>2</b>  |
| <b>1.4 Đối tượng nghiên cứu .....</b>                        | <b>3</b>  |
| <b>1.5 Phương pháp nghiên cứu .....</b>                      | <b>3</b>  |
| 1.5.1 Phương pháp lý thuyết .....                            | 3         |
| 1.5.2 Phương pháp thực nghiệm .....                          | 3         |
| <b>Chương 2: TỔNG QUAN .....</b>                             | <b>4</b>  |
| <b>2.1 Đánh giá các nghiên cứu liên quan.....</b>            | <b>4</b>  |
| <b>2.2 Các hạn chế của nghiên cứu trước đó.....</b>          | <b>4</b>  |
| <b>2.3 Định hướng tiếp cận và giải pháp của đề tài .....</b> | <b>5</b>  |
| <b>Chương 3: CƠ SỞ LÝ THUYẾT .....</b>                       | <b>6</b>  |
| <b>3.1 Trí tuệ nhân tạo .....</b>                            | <b>6</b>  |
| 3.1.1 Giới thiệu về trí tuệ nhân tạo.....                    | 6         |
| 3.1.2 Các nhánh của trí tuệ nhân tạo .....                   | 6         |
| 3.1.3 Các ứng dụng trong thực tế .....                       | 7         |
| <b>3.2 Học máy .....</b>                                     | <b>7</b>  |
| 3.2.1 Giới thiệu về học máy .....                            | 7         |
| 3.2.2 Các phương pháp học máy.....                           | 8         |
| 3.2.3 Quy trình xây dựng mô hình học máy .....               | 8         |
| 3.2.4 Hiện tượng overfitting và underfitting.....            | 8         |
| <b>3.3 Học sâu .....</b>                                     | <b>9</b>  |
| 3.3.1 Giới thiệu về học sâu.....                             | 9         |
| 3.3.2 Mạng nơ-ron tích chập .....                            | 9         |
| 3.3.3 Học chuyển giao.....                                   | 10        |
| <b>3.4 Bài toán Object Detection .....</b>                   | <b>11</b> |
| 3.4.1 Giới thiệu về Object Detection .....                   | 11        |

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| 3.4.2 Các chỉ số đánh giá .....                                  | 12        |
| <b>3.5 YOLO.....</b>                                             | <b>14</b> |
| 3.5.1 Lịch sử phát triển .....                                   | 14        |
| 3.5.2 Giới thiệu về mô hình YOLO26.....                          | 15        |
| 3.5.3 Kiến trúc YOLO26.....                                      | 15        |
| 3.5.4 Các cải tiến chính của YOLO26 .....                        | 17        |
| <b>3.6 Roboflow .....</b>                                        | <b>18</b> |
| 3.6.1 Giới thiệu về Roboflow.....                                | 18        |
| 3.6.2 Tính năng gán nhãn và kho dữ liệu cộng đồng .....          | 19        |
| 3.6.3 Tính năng tiền xử lý và tăng cường dữ liệu .....           | 20        |
| <b>3.7 RESTful API.....</b>                                      | <b>20</b> |
| 3.7.1 Giới thiệu về REST .....                                   | 20        |
| 3.7.2 Các ràng buộc kiến trúc.....                               | 20        |
| 3.7.3 RESTful API .....                                          | 21        |
| <b>3.8 Flask .....</b>                                           | <b>21</b> |
| 3.8.1 Giới thiệu về Flask .....                                  | 21        |
| 3.8.2 Các tính năng.....                                         | 21        |
| <b>3.9 Phát triển ứng dụng di động với Flutter.....</b>          | <b>22</b> |
| 3.9.1 Giới thiệu Flutter.....                                    | 22        |
| 3.9.2 Ngôn ngữ Dart.....                                         | 22        |
| 3.9.3 Ưu nhược điểm của Flutter .....                            | 23        |
| <b>Chương 4: MÔ HÌNH NHẬN DẠNG BỘ CÁNH CỨNG.....</b>             | <b>24</b> |
| <b>4.1 Giới thiệu bộ cánh cứng .....</b>                         | <b>24</b> |
| <b>4.2 Các loại bộ cánh cứng được thu thập trong đề tài.....</b> | <b>24</b> |
| 4.2.1 Bộ dừa .....                                               | 25        |
| 4.2.2 Bộ vòi voi hại dừa .....                                   | 26        |
| 4.2.3 Đuông dừa.....                                             | 28        |
| 4.2.4 Kiến vương.....                                            | 29        |
| 4.2.5 Bộ hà khoai lang.....                                      | 30        |
| 4.2.6 Bộ rầy / Bù rầy .....                                      | 31        |
| 4.2.7 Câu cầu xanh .....                                         | 32        |
| 4.2.8 Bộ nhảy sọc cong .....                                     | 33        |
| 4.2.9 Bộ ngẫu .....                                              | 34        |

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>4.3 Quy trình xây dựng mô hình nhận diện bộ cánh cứng .....</b> | <b>35</b> |
| <b>4.4 Thu thập dữ liệu.....</b>                                   | <b>36</b> |
| 4.4.1 Nguồn dữ liệu.....                                           | 36        |
| 4.4.2 Số lượng và phân phối dữ liệu theo lớp .....                 | 37        |
| <b>4.5 Xử lý dữ liệu .....</b>                                     | <b>38</b> |
| 4.5.1 Quy trình đóng khung và gán nhãn .....                       | 38        |
| 4.5.2 Chia dữ liệu .....                                           | 40        |
| 4.5.3 Tăng cường dữ liệu .....                                     | 41        |
| <b>4.6 Huấn luyện mô hình .....</b>                                | <b>43</b> |
| 4.6.1 Môi trường huấn luyện.....                                   | 43        |
| 4.6.2 Huấn luyện mô hình .....                                     | 44        |
| <b>4.7 Kết quả huấn luyện.....</b>                                 | <b>45</b> |
| 4.7.1 Kết quả tổng thể .....                                       | 45        |
| 4.7.2 Kết quả theo từng loài .....                                 | 45        |
| 4.7.3 Các biểu đồ kết quả .....                                    | 46        |
| 4.7.4 Nhận xét .....                                               | 51        |
| <b>Chương 5: XÂY DỰNG HỆ THỐNG NHẬN DIỆN BỘ CÁNH CỨNG .....</b>    | <b>52</b> |
| <b>5.1 Yêu cầu chức năng .....</b>                                 | <b>52</b> |
| <b>5.2 Kiến trúc hệ thống .....</b>                                | <b>52</b> |
| 5.2.1 Các thành phần chính .....                                   | 53        |
| 5.2.2 Luồng hoạt động của hệ thống .....                           | 53        |
| <b>5.3 Hoạt động chức năng hệ thống .....</b>                      | <b>55</b> |
| <b>5.4 Thiết kế cơ sở dữ liệu.....</b>                             | <b>56</b> |
| <b>5.5 Thiết kế API.....</b>                                       | <b>56</b> |
| <b>Chương 6: KẾT QUẢ THỰC HIỆN.....</b>                            | <b>59</b> |
| <b>6.1 Giao diện ứng dụng.....</b>                                 | <b>59</b> |
| 6.1.1 Trang chủ ứng dụng .....                                     | 59        |
| 6.1.2 Trang chọn dữ liệu tải lên.....                              | 60        |
| 6.1.3 Trang nhận diện ảnh .....                                    | 61        |
| 6.1.4 Trang xử lý video .....                                      | 61        |
| 6.1.5 Trang kết quả nhận diện video .....                          | 62        |
| 6.1.6 Trang danh sách loài.....                                    | 64        |
| <b>6.2 Kết quả nhận diện thực tế.....</b>                          | <b>64</b> |

|                                                     |           |
|-----------------------------------------------------|-----------|
| 6.2.1 Kết quả nhận diện trên ảnh.....               | 64        |
| 6.2.2 Kết quả nhận diện trên video .....            | 65        |
| <b>6.3 Đánh giá hiệu năng ứng dụng.....</b>         | <b>66</b> |
| <b>Chương 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN .....</b> | <b>67</b> |
| <b>7.1 Kết luận.....</b>                            | <b>67</b> |
| 7.1.1 Ưu điểm.....                                  | 67        |
| 7.1.2 Nhược điểm.....                               | 67        |
| <b>7.2 Hướng phát triển.....</b>                    | <b>68</b> |
| <b>PHỤ LỤC .....</b>                                | <b>69</b> |
| <b>DANH MỤC TÀI LIỆU THAM KHẢO .....</b>            | <b>70</b> |



## DANH MỤC HÌNH ẢNH

|           |                                                          |    |
|-----------|----------------------------------------------------------|----|
| Hình 3.1  | Họ các mô hình nhận diện đối tượng .....                 | 11 |
| Hình 3.2  | Công thức tính IoU .....                                 | 12 |
| Hình 3.3  | Kiến trúc YOLO26 .....                                   | 16 |
| Hình 4.1  | Bộ dũa.....                                              | 25 |
| Hình 4.2  | Tác hại của bộ dũa.....                                  | 26 |
| Hình 4.3  | Bộ vòi voi hại dũa .....                                 | 27 |
| Hình 4.4  | Tác hại của bộ vòi voi.....                              | 27 |
| Hình 4.5  | Đuông dũa .....                                          | 28 |
| Hình 4.6  | Tác hại của đuông dũa.....                               | 29 |
| Hình 4.7  | Kiến vương .....                                         | 29 |
| Hình 4.8  | Bộ hà khoai láng.....                                    | 30 |
| Hình 4.9  | Bộ rầy .....                                             | 31 |
| Hình 4.10 | Câu câu xanh .....                                       | 32 |
| Hình 4.11 | Bộ nhảy sọc cong.....                                    | 33 |
| Hình 4.12 | Bộ ngẫu.....                                             | 34 |
| Hình 4.13 | Quy trình xây dựng mô hình nhận diện.....                | 35 |
| Hình 4.14 | Giao diện trang chủ Roboflow .....                       | 38 |
| Hình 4.15 | Khởi tạo dự án Object Detection trên Roboflow.....       | 39 |
| Hình 4.16 | Tải dữ liệu lên Roboflow.....                            | 39 |
| Hình 4.17 | Gán nhãn đối tượng bằng bounding box trên Roboflow ..... | 40 |
| Hình 4.18 | Chia dữ liệu .....                                       | 41 |
| Hình 4.19 | Chọn phép tăng cường dữ liệu.....                        | 41 |
| Hình 4.20 | Ảnh dùng Flip.....                                       | 42 |
| Hình 4.21 | Ảnh dùng Crop .....                                      | 42 |
| Hình 4.22 | Ảnh dùng Rotation .....                                  | 43 |
| Hình 4.23 | Biểu đồ đường cong Precision–Recall .....                | 46 |
| Hình 4.24 | Biểu đồ đường cong F1 - Confidence .....                 | 47 |
| Hình 4.25 | Ma trận nhầm lẫn.....                                    | 48 |
| Hình 4.26 | Biểu đồ kết quả theo từng epoch .....                    | 49 |
| Hình 4.27 | Ảnh được gán nhãn.....                                   | 50 |
| Hình 4.28 | Ảnh kết quả dự đoán.....                                 | 50 |

|                                             |    |
|---------------------------------------------|----|
| Hình 5.1 Kiến trúc hệ thống .....           | 53 |
| Hình 5.2 Lưu đồ chức năng hệ thống .....    | 55 |
| Hình 6.1 Giao diện trang chủ .....          | 59 |
| Hình 6.2 Trang chọn ảnh từ thư viện.....    | 60 |
| Hình 6.3 Trang kết quả nhận diện ảnh.....   | 61 |
| Hình 6.4 Trang chờ xử lý.....               | 62 |
| Hình 6.5 Kết quả nhận diện trên video ..... | 63 |
| Hình 6.6 Trang danh sách loài .....         | 64 |
| Hình 6.7 Ảnh gốc.....                       | 65 |
| Hình 6.8 Ảnh nhận diện.....                 | 65 |
| Hình 6.9 Ảnh từ video gốc .....             | 65 |
| Hình 6.10 Ảnh từ video nhận diện .....      | 65 |

## DANH MỤC BẢNG BIỂU

|                                                  |    |
|--------------------------------------------------|----|
| Bảng 4-1 Phân phối dữ liệu theo lớp .....        | 37 |
| Bảng 4-2 Cấu hình môi trường huấn luyện .....    | 44 |
| Bảng 4-3 Kết quả huấn luyện tổng thể .....       | 45 |
| Bảng 4-4 Kết quả huấn luyện theo từng loài ..... | 45 |
| Bảng 5-1 Bảng dữ liệu species .....              | 56 |
| Bảng 5-2 Tổng hợp các API sử dụng.....           | 57 |

## KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT

| Viết tắt | Chữ đầy đủ                        | Diễn giải                             |
|----------|-----------------------------------|---------------------------------------|
| AI       | Artificial Intelligence           | Trí tuệ nhân tạo                      |
| API      | Application Programming Interface | Giao diện lập trình ứng dụng          |
| CNN      | Convolutional Neural Network      | Mạng nơ-ron tích chập                 |
| COCO     | Common Objects in Context         | Bộ dữ liệu ảnh COCO                   |
| CORS     | Cross-Origin Resource Sharing     | Chia sẻ tài nguyên giữa các nguồn gốc |
| DL       | Deep Learning                     | Học sâu                               |
| EXIF     | Exchangeable Image File Format    | Định dạng dữ liệu kèm theo ảnh EXIF   |
| IoU      | Intersection over Union           | Tỉ lệ giao trên hợp                   |
| JSON     | JavaScript Object Notation        | Ký pháp đối tượng JavaScript          |
| ML       | Machine Learning                  | Học máy                               |
| mAP      | Mean Average Precision            | Độ chính xác trung bình               |
| ONNX     | Open Neural Network Exchange      | Định dạng trao đổi mạng nơ-ron mở     |
| ORM      | Object-Relational Mapping         | Ảnh xạ quan hệ - đối tượng            |
| REST     | Representational State Transfer   | Chuyển giao trạng thái đại diện       |
| YOLO     | You Only Look Once                | Mô hình phát hiện đối tượng YOLO      |

## **Chương 1: MỞ ĐẦU**

Chương này trình bày về lý do chọn đề tài, mục tiêu của đề tài, đối tượng nghiên cứu, phạm vi nghiên cứu và phương pháp nghiên cứu. Nhằm giới thiệu rõ về phạm vi của đề án cũng như cách giải quyết vấn đề.

### **1.1 Lý do chọn đề tài**

Trí tuệ nhân tạo đã trở thành những lĩnh vực thu hút sự quan tâm lớn từ cộng đồng học thuật và sinh viên. Việc ứng dụng các kỹ thuật này vào các bài toán phân loại và nhận diện đối tượng trong tự nhiên là một hướng nghiên cứu có giá trị thực tiễn cao, đồng thời cũng là môi trường lý tưởng để rèn luyện các kỹ năng cốt lõi như xây dựng và tiền xử lý bộ dữ liệu, huấn luyện và đánh giá mô hình, tích hợp mô hình vào ứng dụng thực tế.

Bộ cánh cứng là bộ côn trùng đa dạng sở hữu hình thái phong phú. Đây là đối tượng phù hợp để nghiên cứu ứng dụng các mô hình phát hiện đối tượng hiện đại, vì dữ liệu hình ảnh đa dạng và phong phú, có thể thu thập từ các nền tảng cộng đồng như iNaturalist.

Flutter là nền tảng phát triển ứng dụng di động đa nền tảng được Google phát triển, ngày càng phổ biến trong cộng đồng lập trình viên. Cùng với đó việc tích hợp mô hình AI vào ứng dụng Flutter thông qua REST API là kỹ năng thực tế có giá trị cao trong việc phát triển phần mềm hiện nay.

Nhận biết một loài côn trùng gây hại trên đồng ruộng phần lớn vẫn dựa vào kinh nghiệm của người trồng hoặc phải tra cứu trong các tài liệu chuyên ngành. Cách làm này mất thời gian, đòi hỏi kiến thức nhất định và không phải lúc nào cũng cho kết quả chính xác, nhất là khi nhiều loài có hình dạng gần giống nhau. Điện thoại thông minh ngày nay gần như đã trở thành vật dụng quen thuộc của nhiều người nên một ứng dụng nhận diện bộ cánh cứng chạy trên điện thoại là hoàn toàn hợp lý và có tính ứng dụng cao.

Xuất phát từ mong muốn vận dụng kiến thức đã học vào một bài toán cụ thể, đề tài xây dựng ứng dụng nhận diện bộ cánh cứng được thực nhằm mục đích học tập và nghiên cứu, tìm hiểu quy trình xây dựng một hệ thống thị giác máy tính hoàn chỉnh từ thu thập và xử lý dữ liệu, huấn luyện mô hình học sâu, đến triển khai vào ứng dụng di động thực tế.

### **1.2 Mục tiêu đề tài**

Thu thập và xây dựng bộ dữ liệu hình ảnh bọ cánh cứng từ nhiều nguồn khác nhau như iNaturalist, Google Images, Roboflow Universe và ảnh tự thu thập. Bộ dữ liệu được xây dựng với yêu cầu bảo đảm tính đa dạng về góc độ chụp, điều kiện ánh sáng và môi trường nền, bởi đây là yếu tố quan trọng giúp mô hình học được đặc trưng của từng loài trong nhiều tình huống thực tế khác nhau, qua đó hạn chế hiện tượng nhận diện sai khi gặp ảnh có điều kiện chụp không lý tưởng.

Huấn luyện và tối ưu mô hình YOLO26 trên bộ dữ liệu tự xây dựng. Quá trình huấn luyện được thực hiện trên môi trường Google Colab, kết hợp kỹ thuật học chuyển giao nhằm tận dụng trọng số đã được huấn luyện trước và rút ngắn thời gian huấn luyện. Mục tiêu đặt ra là mô hình đạt chỉ số mAP50 từ 0,80 trở lên trên tập kiểm tra, đồng thời cân bằng giữa độ chính xác và tốc độ xử lý để phù hợp với việc triển khai trên thiết bị di động.

Xây dựng RESTful API backend bằng Python Flask đóng vai trò trung gian giữa ứng dụng và mô hình. Backend có nhiệm vụ tiếp nhận ảnh do người dùng gửi lên, đưa ảnh qua mô hình YOLO26 để nhận diện, truy xuất thông tin chi tiết của loài tương ứng từ cơ sở dữ liệu MySQL và trả kết quả về cho ứng dụng dưới định dạng JSON.

Phát triển ứng dụng di động đa nền tảng bằng Flutter làm giao diện tương tác với người dùng. Ứng dụng cung cấp các chức năng chính gồm chụp ảnh và quay video hoặc chọn từ thư viện, gửi ảnh lên API để nhận diện, hiển thị kết quả kèm theo thông tin về loài, mức độ gây hại và biện pháp phòng chống.

Kiểm thử và đánh giá toàn bộ hệ thống trong điều kiện thực tế trên thiết bị chạy hệ điều hành Android. Việc kiểm thử nhằm xác minh khả năng hoạt động ổn định của các thành phần khi phối hợp với nhau, đồng thời đánh giá độ chính xác của kết quả nhận diện và khả năng ứng dụng của hệ thống vào thực tế.

### **1.3 Phạm vi nghiên cứu**

Đề tài tập trung vào nhận diện một số loài bọ cánh cứng tại Đồng bằng sông Cửu Long. Phạm vi không bao gồm toàn bộ các họ trong bộ cánh cứng mà chỉ giới hạn ở một số lớp loài được lựa chọn dựa trên khả năng thu thập dữ liệu. Ứng dụng được phát triển và kiểm thử trên thiết bị di động chạy hệ điều hành Android.

---

### **1.4 Đối tượng nghiên cứu**

Đề tài tập trung nghiên cứu các nhóm đối tượng chính sau. Các loài bọ cánh cứng thuộc bộ Coleoptera. Mô hình học sâu YOLO26 cùng với kỹ thuật transfer learning. Nghiên cứu quy trình xây dựng bộ dữ liệu hoàn chỉnh bao gồm các bước thu thập ảnh từ nhiều nguồn, làm sạch và lọc ảnh kém chất lượng, gán nhãn và tăng cường dữ liệu để mở rộng tập huấn luyện. Cuối cùng là xây dựng ứng dụng di động đa nền tảng Flutter cung cấp giao diện cho người dùng chụp ảnh và hiển thị kết quả nhận diện một cách trực quan.

### **1.5 Phương pháp nghiên cứu**

#### **1.5.1 Phương pháp lý thuyết**

Nghiên cứu, tổng hợp tài liệu về các lĩnh vực liên quan bao gồm đặc điểm sinh học, tác hại và biện pháp phòng chống của một loài bọ cánh cứng gây hại ở khu vực Đồng bằng sông Cửu Long; các nhánh của trí tuệ nhân tạo gồm học máy, học sâu và thị giác máy tính; kiến trúc mạng nơ-ron tích chập; nguyên lý hoạt động và kiến trúc của mô hình YOLO26; phương pháp xây dựng RESTful API với Flask; phát triển ứng dụng di động đa nền tảng với Flutter; và quy trình xây dựng, quản lý bộ dữ liệu thị giác máy tính với Roboflow.

#### **1.5.2 Phương pháp thực nghiệm**

Tiến hành thực nghiệm theo quy trình: thu thập và xây dựng bộ dữ liệu thực tế; gán nhãn và tiền xử lý dữ liệu trên Roboflow; huấn luyện, đánh giá và tối ưu mô hình YOLO26 trên Google Colab có hỗ trợ GPU; xây dựng backend Flask API kết hợp cơ sở dữ liệu MySQL và phát triển ứng dụng Flutter; cuối cùng là kiểm thử, đánh giá toàn bộ hệ thống trên thiết bị thực tế.

## **Chương 2: TỔNG QUAN**

Việc ứng dụng học sâu vào nhận dạng côn trùng và sâu hại đã được nhiều nhóm nghiên cứu trong và ngoài nước quan tâm. Chương này điếm qua một số công trình tiêu biểu, phân tích đặc điểm nổi bật cùng hạn chế của chúng, từ đó xác định hướng nghiên cứu của đề tài sao cho không trùng lặp và có tính cải tiến.

### **2.1 Đánh giá các nghiên cứu liên quan**

Nhận dạng côn trùng và sâu hại bằng học sâu là một hướng nghiên cứu đã thu hút sự quan tâm của nhiều nhóm tác giả trong và ngoài nước, với hai cách tiếp cận chính là phân loại bằng mạng nơ-ron tích chập và phát hiện đối tượng bằng họ mô hình YOLO.

Hansen và cộng sự (2020) là một trong những nghiên cứu tiêu biểu áp dụng mạng nơ-ron tích chập để phân loại côn trùng đến cấp loài. Nhóm tác giả xây dựng cơ sở dữ liệu gồm 65.841 ảnh mẫu vật, bao phủ 361 loài bọ chân chạy. Trên cơ sở đó, họ tinh chỉnh một mạng CNN đã được huấn luyện trước để phân loại ảnh chụp toàn thân mẫu vật nhìn từ mặt lưng. Do mẫu vật được đặt trong khay theo tư thế chuẩn với đầu cùng một hướng, mức độ biến thiên của dữ liệu được giảm thiểu. Mô hình đạt độ chính xác cao và phân loại được nhiều loài đến cấp chi và loài [12].

Tại Việt Nam, các nghiên cứu chủ yếu hướng đến sâu rầy và bệnh hại trên lúa cùng một số cây trồng. Hệ thống T-Pest sử dụng mô hình cải tiến YOLOv5-Ghost kết hợp máy thu thập dữ liệu tự động và nền tảng web để giám sát dịch hại theo thời gian thực; hệ thống của Rynan nhận diện được khoảng 120 loại côn trùng và hiển thị kết quả trên điện thoại [6]. Bên cạnh đó, nhiều nghiên cứu dùng YOLOv4, YOLOv9 cho nhận dạng bệnh trên dưa chuột, cây lúa cũng cho kết quả tích cực.

### **2.2 Các hạn chế của nghiên cứu trước đó**

Mặc dù đã có nhiều nghiên cứu về nhận dạng côn trùng và sâu hại, vẫn còn tồn tại một số hạn chế cần tiếp tục cải thiện. Bộ cánh cứng còn ít được khai thác riêng trong các nghiên cứu học sâu, một phần do thiếu các tập ảnh công khai, trong khi phần lớn công trình trong nước lại tập trung vào sâu rầy và bệnh hại trên lúa hơn là các loài bọ cánh cứng. Nhiều hệ thống đòi hỏi phần cứng chuyên dụng như bẫy đèn, camera cố định và cảm biến, hoặc được triển khai trên nền tảng web, khiến chúng khó tiếp cận với người dùng phổ thông.



## ***Xây dựng ứng dụng nhận diện bọ cánh cứng***

---

Ngoài ra, phần lớn nghiên cứu mới dừng ở khâu phát hiện hoặc phân loại mà chưa cung cấp kèm theo thông tin sinh học hữu ích như đặc điểm, tác hại và biện pháp phòng trừ, đồng thời các công cụ nhận diện bọ cánh cứng hỗ trợ tốt tiếng Việt cho người dùng trong nước vẫn còn hạn chế. Những hạn chế này cho thấy vẫn còn nhiều cơ hội để xây dựng một ứng dụng nhận diện bọ cánh cứng hoạt động được trên thiết bị di động phổ thông, đồng thời cung cấp thông tin hữu ích cho người dùng.

### **2.3 Định hướng tiếp cận và giải pháp của đề tài**

Có thể thấy học sâu, đặc biệt là họ mô hình YOLO, đã chứng minh hiệu quả trong nhận dạng côn trùng và cho phép phát hiện theo thời gian thực. Tuy nhiên, các nghiên cứu trước vẫn còn một số hạn chế.

Trên cơ sở đó, đề tài đề xuất một hướng nghiên cứu riêng có tính cải tiến so với các công trình trước. Đề tài hướng tới tập trung vào các loài bọ cánh cứng gây hại phổ biến ở Việt Nam nhằm góp phần hỗ trợ cho người nông dân trong khu vực. Về mô hình, đề tài dự kiến sử dụng YOLO26 phiên bản mới hơn so với các phiên bản được dùng trong phần lớn nghiên cứu trước nhằm cân bằng giữa độ chính xác và khả năng triển khai nhẹ. Hệ thống được định hướng phát triển thành ứng dụng di động đa nền tảng bằng Flutter, hoạt động trên điện thoại thông minh mà không cần bất kỳ đèn hay cảm biến chuyên dụng như các hệ thống trong nước đồng thời ứng dụng dự kiến hỗ trợ nhận diện trên cả ảnh và video thay vì chỉ một dạng dữ liệu tích hợp cơ sở dữ liệu thông tin loài gồm đặc điểm sinh học, tác hại và biện pháp phòng trừ, giúp người dùng không chỉ có thể nhận diện được loài mà còn hiểu cách xử lý phù hợp.

### **Chương 3: CƠ SỞ LÝ THUYẾT**

Chương này trình bày các cơ sở lý thuyết nền tảng cho việc xây dựng ứng dụng nhận diện bọ cánh cứng, được tổ chức theo hai nhóm. Nhóm thứ nhất hướng đến mô hình nhận diện, gồm các khái niệm về trí tuệ nhân tạo, học máy, học sâu, bài toán phát hiện đối tượng, họ mô hình YOLO và công cụ Roboflow phục vụ gán nhãn dữ liệu. Nhóm thứ hai đề cập các công nghệ triển khai ứng dụng, gồm RESTful API và Flask ở phía máy chủ, cùng Flutter ở phía người dùng. Những lý thuyết này là cơ sở cho các lựa chọn kỹ thuật được áp dụng ở những chương tiếp theo.

#### **3.1 Trí tuệ nhân tạo**

##### **3.1.1 Giới thiệu về trí tuệ nhân tạo**

Trí tuệ nhân tạo (AI) là lĩnh vực của khoa học máy tính nghiên cứu việc xây dựng các hệ thống có khả năng thực hiện những nhiệm vụ vốn đòi hỏi trí tuệ của con người, chẳng hạn như nhận thức, suy luận, học hỏi và ra quyết định. Các hệ thống trí tuệ nhân tạo hiện đại được thiết kế để tiếp nhận dữ liệu, rút ra quy luật và đưa ra phản hồi phù hợp với ngữ cảnh mà không cần mô tả trước toàn bộ các trường hợp có thể xảy ra.

Trí tuệ nhân tạo là một lĩnh vực rộng, bao hàm nhiều hướng tiếp cận khác nhau. Trong giai đoạn đầu, các hệ thống AI chủ yếu dựa trên tập luật và tri thức do con người định nghĩa. Về sau, các phương pháp học từ dữ liệu, đặc biệt là học máy và học sâu, dần trở thành hướng tiếp cận chủ đạo nhờ khả năng tự cải thiện hiệu năng theo dữ liệu và giải quyết hiệu quả nhiều bài toán phức tạp trong thực tế [18].

##### **3.1.2 Các nhánh của trí tuệ nhân tạo**

Lĩnh vực AI bao gồm nhiều nhánh nghiên cứu quan trọng, mỗi nhánh tập trung vào một khía cạnh khác nhau của trí thông minh máy tính.

Học máy (Machine Learning) là nhánh cho phép máy tính học từ dữ liệu và cải thiện hiệu suất theo thời gian mà không cần được lập trình cụ thể từng quy tắc. Thay vì viết code để xử lý từng trường hợp, người ta cung cấp dữ liệu và để thuật toán tự tìm ra các mẫu. ML bao gồm ba hướng tiếp cận chính học có giám sát là học từ dữ liệu đã được gán nhãn, học không giám sát tự khám phá cấu trúc trong dữ liệu chưa có nhãn và học tăng cường [14].

Học sâu (Deep Learning) là một nhánh của học máy, sử dụng mạng nơ-ron nhân tạo có nhiều lớp ẩn để tự động học các biểu diễn đặc trưng phân cấp từ dữ liệu thô. Đây là nhánh phát triển mạnh mẽ nhất và có nhiều đột phá nhất trong thập kỷ qua, đặc biệt trong các lĩnh vực thị giác máy tính và xử lý ngôn ngữ tự nhiên [10].

Thị giác máy tính (Computer Vision) là nhánh nghiên cứu cho phép máy tính phân tích, hiểu và trích xuất thông tin có ý nghĩa từ hình ảnh và video, bao gồm các tác vụ như phân loại ảnh, phát hiện đối tượng, phân đoạn ảnh và ước lượng tư thế [19].

### **3.1.3 Các ứng dụng trong thực tế**

AI hiện nay được ứng dụng rộng rãi trong hầu hết mọi lĩnh vực của đời sống và sản xuất. Trong y tế, các hệ thống AI có khả năng phân tích ảnh X-quang, MRI để hỗ trợ chẩn đoán bệnh với độ chính xác tương đương bác sĩ chuyên khoa. Trong giao thông, công nghệ xe tự lái kết hợp nhiều nhánh AI để nhận diện biển báo, phương tiện, người đi bộ và ra quyết định lái xe an toàn. Trong thương mại điện tử, các hệ thống gợi ý sản phẩm dựa trên hành vi người dùng giúp tăng tỷ lệ chuyển đổi đáng kể. Trong lĩnh vực nông nghiệp AI đang được ứng dụng ngày càng rộng rãi để giám sát cây trồng qua ảnh vệ tinh, phát hiện sớm dịch bệnh và sâu hại, dự báo năng suất và tối ưu hóa lịch bón phân.

## **3.2 Học máy**

### **3.2.1 Giới thiệu về học máy**

Học máy là một nhánh của trí tuệ nhân tạo, nghiên cứu các thuật toán cho phép máy tính học các quy luật từ dữ liệu và đưa ra dự đoán hoặc quyết định mà không cần được lập trình tường minh cho từng trường hợp cụ thể. Một hệ thống ML được cung cấp một tập dữ liệu, từ đó tự xác định các mẫu và xây dựng mô hình ánh xạ giữa đầu vào và đầu ra [14].

Năng lực cốt lõi của ML nằm ở khả năng khái quát hóa (generalization) tức là khả năng hoạt động chính xác trên những dữ liệu mới chưa từng xuất hiện trong quá trình huấn luyện. Vì vậy, chất lượng và quy mô của dữ liệu huấn luyện là một trong những yếu tố quyết định đến hiệu năng của mô hình dữ liệu càng đa dạng, đầy đủ và được gán nhãn chính xác thì mô hình càng có khả năng khái quát hóa tốt.

### **3.2.2 Các phương pháp học máy**

Dựa trên đặc điểm của dữ liệu và cách thức học, học máy được phân thành ba nhóm phương pháp chính như sau:

Học có giám sát (Supervised Learning) mô hình được huấn luyện trên tập dữ liệu đã được gán nhãn, trong đó mỗi mẫu đầu vào tương ứng với một kết quả đầu ra mong muốn. Mục tiêu của quá trình huấn luyện là học được hàm ánh xạ từ đầu vào sang đầu ra để dự đoán nhãn cho dữ liệu mới. Học có giám sát bao gồm hai dạng bài toán chính là phân loại (classification) và hồi quy (regression), khi đầu ra là giá trị liên tục.

Học không giám sát (Unsupervised Learning) mô hình được huấn luyện trên dữ liệu chưa gán nhãn nhằm khám phá cấu trúc tiềm ẩn của dữ liệu, chẳng hạn như phân cụm (clustering) các mẫu có đặc điểm tương tự hoặc giảm số chiều của dữ liệu.

Học tăng cường (Reinforcement Learning) mô hình học thông qua quá trình tương tác với môi trường và nhận về tín hiệu phản hồi dưới dạng phần thưởng hoặc hình phạt sau mỗi hành động, từ đó dần điều chỉnh chiến lược nhằm tối đa hóa phần thưởng tích lũy.

### **3.2.3 Quy trình xây dựng mô hình học máy**

Quá trình xây dựng một mô hình học máy nói chung thường trải qua các bước cơ bản như sau: thu thập và chuẩn bị dữ liệu; tiền xử lý dữ liệu như làm sạch, chuẩn hóa, gán nhãn; phân chia dữ liệu thành tập huấn luyện, tập kiểm định và tập kiểm tra; huấn luyện mô hình trên tập huấn luyện; đánh giá mô hình trên tập kiểm tra bằng các chỉ số phù hợp; và cuối cùng là triển khai mô hình vào ứng dụng thực tế. Việc phân chia dữ liệu thành các tập riêng biệt nhằm bảo đảm mô hình được đánh giá một cách khách quan trên những dữ liệu chưa từng được sử dụng trong huấn luyện. Quy trình thực nghiệm của đề tài cũng được tổ chức theo trình tự này.

### **3.2.4 Hiện tượng overfitting và underfitting**

Trong quá trình huấn luyện, hai hiện tượng thường gặp ảnh hưởng đến khả năng khái quát hóa của mô hình là overfitting và underfitting. Overfitting xảy ra khi mô hình học quá chi tiết tập huấn luyện, kể cả nhiễu, dẫn đến hoạt động tốt trên dữ liệu huấn luyện nhưng kém trên dữ liệu mới. Ngược lại, underfitting xảy ra khi mô hình quá đơn giản, không nắm bắt được quy luật của dữ liệu, dẫn đến hiệu năng thấp trên cả tập huấn luyện lẫn tập kiểm

tra. Một số kỹ thuật thường được sử dụng để hạn chế quá khớp gồm tăng cường dữ liệu, điều chuẩn và dừng sớm.

### **3.3 Học sâu**

#### **3.3.1 Giới thiệu về học sâu**

Học sâu sử dụng các mạng nơ-ron nhân tạo có nhiều lớp ẩn gọi là mạng nơ-ron sâu để tự động học các biểu diễn đặc trưng có tính phân cấp từ dữ liệu thô. Goodfellow, Bengio và Courville định nghĩa học sâu là phương pháp học biểu diễn cho phép máy tính học từ kinh nghiệm và hiểu thế giới theo cấu trúc phân cấp khái niệm, trong đó mỗi khái niệm được định nghĩa thông qua mối quan hệ với các khái niệm đơn giản hơn [10].

Điểm khác biệt cơ bản giữa học sâu và học máy truyền thống nằm ở khâu trích xuất đặc trưng. Với học máy truyền thống các kỹ sư phải thiết kế đặc trưng thủ công tức là phải tự xác định và tính toán những đặc điểm nào của dữ liệu là quan trọng để đưa vào mô hình. Học sâu loại bỏ bước tốn kém này bằng cách để mạng nơ-ron tự động học đặc trưng trực tiếp từ dữ liệu thô thông qua quá trình huấn luyện. Nhờ đó học sâu đặc biệt hiệu quả với các loại dữ liệu phi cấu trúc như hình ảnh, âm thanh và văn bản.

Quá trình huấn luyện mạng nơ-ron sâu sử dụng thuật toán lan truyền ngược kết hợp với các biến thể của gradient descent để tối ưu hàm mất mát. Cụ thể, sau mỗi lần dự đoán, sai số giữa kết quả dự đoán và nhãn thực tế được tính toán, sau đó được lan truyền ngược từ lớp đầu ra về lớp đầu vào để cập nhật trọng số của toàn mạng theo hướng giảm thiểu sai số. Quá trình này lặp đi lặp lại qua hàng chục đến hàng trăm nghìn vòng trên tập dữ liệu huấn luyện cho đến khi mô hình hội tụ.

#### **3.3.2 Mạng nơ-ron tích chập**

Mạng nơ-ron tích chập (Convolutional Neural Network) là một kiến trúc học sâu được thiết kế chuyên biệt để xử lý dữ liệu có cấu trúc dạng lưới điển hình là hình ảnh. CNN khai thác hai đặc tính quan trọng của dữ liệu ảnh là tính cục bộ và tính bất biến với phép dịch chuyển. CNN trích xuất đặc trưng hiệu quả hơn và cần ít tham số hơn so với mạng nơ-ron kết nối đầy đủ. Hiện nay, CNN là nền tảng kiến trúc của hầu hết các mô hình thị giác máy tính bao gồm các mô hình phát hiện đối tượng như YOLO [10][15].

## ***Xây dựng ứng dụng nhận diện bộ cánh cứng***

---

Một mạng CNN điển hình được cấu thành từ ba loại lớp chính:

Lớp tích chập (Convolutional Layer) là thành phần cốt lõi của CNN, sử dụng các bộ lọc (kernel) kích thước nhỏ trượt trên ảnh đầu vào để tính tích chập và tạo ra các bản đồ đặc trưng (feature map). Các bộ lọc ở những lớp đầu thường học các đặc trưng đơn giản như cạnh, trong khi các lớp sâu hơn học các đặc trưng phức tạp hơn như hình dạng. Cơ chế chia sẻ trọng số giúp giảm đáng kể số lượng tham số cần học.

Hàm kích hoạt phi tuyến (Activation Function) được áp dụng sau lớp tích chập nhằm bổ sung tính phi tuyến cho mô hình.

Lớp gộp (Pooling Layer) thực hiện giảm kích thước không gian của bản đồ đặc trưng, qua đó giảm khối lượng tính toán và tăng tính bất biến của mô hình đối với các biến đổi nhỏ của ảnh.

Sau các khối tích chập và gộp, đặc trưng được tổng hợp và đưa qua phần đầu của mạng để đưa ra dự đoán. Trong các mô hình phát hiện đối tượng hiện đại như YOLO, phần đầu dự đoán được thiết kế để xuất đồng thời thông tin về vị trí và lớp của đối tượng.

### **3.3.3 Học chuyển giao**

Học chuyển giao (Transfer Learning) là kỹ thuật tận dụng kiến thức dưới dạng trọng số mạng nơ-ron đã được học từ một tác vụ hoặc bộ dữ liệu lớn để giải quyết hiệu quả hơn một tác vụ mới với ít dữ liệu hơn.

Học chuyển giao thường được thực hiện theo quy trình đầu tiên khởi tạo mô hình với trọng số đã được huấn luyện trước trên một bộ dữ liệu lớn như ImageNet hoặc COCO sau đó tinh chỉnh mô hình trên bộ dữ liệu chuyên biệt của tác vụ đích. Tùy theo độ tương đồng giữa hai bài toán và lượng dữ liệu có sẵn có thể chọn tinh chỉnh toàn bộ mạng, chỉ tinh chỉnh các lớp cao hoặc áp dụng learning rate khác nhau cho từng phần của mạng.

Lợi ích là giảm đáng kể lượng dữ liệu cần thiết thay vì cần hàng triệu ảnh để huấn luyện từ đầu thì chỉ cần số lượng ít ảnh là đã đạt được kết quả tốt, rút ngắn thời gian huấn luyện. Cải thiện độ chính xác mô hình bằng việc tận dụng được đặc trưng thị giác phổ quát đã học từ COCO [10][21].

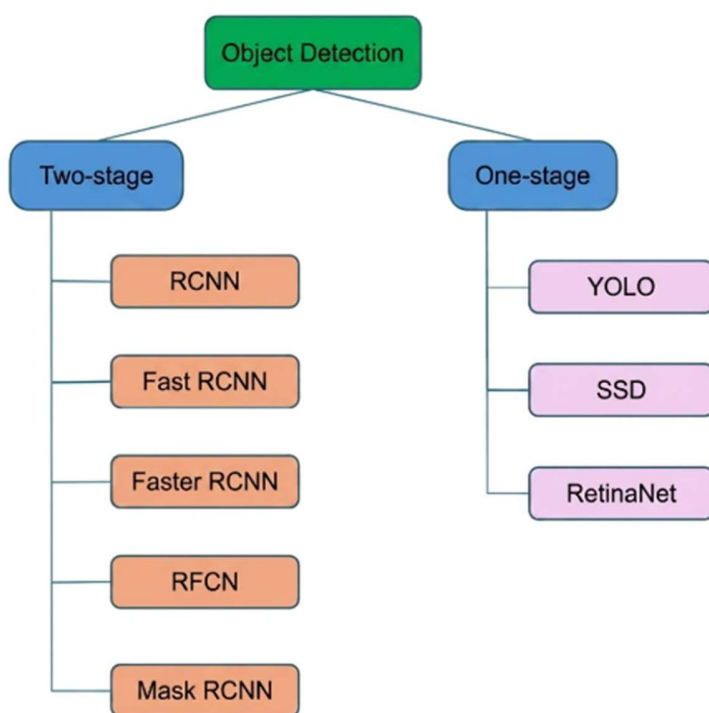
### 3.4 Bài toán Object Detection

#### 3.4.1 Giới thiệu về Object Detection

Đây là một trong những bài toán trung tâm và thách thức nhất của thị giác máy tính. Khác với bài toán phân loại ảnh (image classification), phát hiện đối tượng phải đồng thời giải quyết hai bài toán xác định vị trí chính xác của từng đối tượng trong ảnh bằng cách dự đoán bounding box bao quanh nó và phân loại xem đối tượng đó thuộc lớp nào.

Một bounding box thường được biểu diễn bởi bốn giá trị tọa độ tâm (cx, cy), chiều rộng (w) và chiều cao (h) hoặc tọa độ góc trên-trái và góc dưới-phải (x1, y1, x2, y2). Trong định dạng YOLO, các giá trị này được chuẩn hóa về khoảng  $[0, 1]$  so với kích thước ảnh để độc lập với độ phân giải đầu vào [21].

Hình 2.1 giới thiệu về các mô hình nhận diện đối tượng, các mô hình này được phân thành hai nhóm chính phương pháp hai giai đoạn (two-stage detectors) và phương pháp một giai đoạn (one-stage detectors).



Hình 3.1 Họ các mô hình nhận diện đối tượng

Phương pháp hai giai đoạn, tiêu biểu là dòng R-CNN và Faster R-CNN, thực hiện nhận diện qua hai bước tuần tự. Mạng đề xuất vùng (Region Proposal Network) sinh ra các

---

### ***Xây dựng ứng dụng nhận diện bọ cánh cứng***

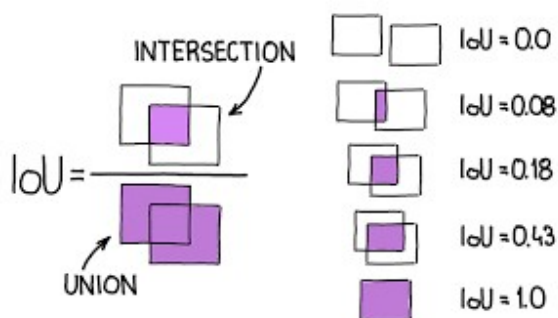
vùng ứng viên có khả năng chứa đối tượng sau đó mô hình trích xuất đặc trưng và phân loại cho từng vùng đề xuất. Cách tiếp cận này đạt độ chính xác cao nhưng tốc độ suy luận chậm, do đó ít phù hợp với các ứng dụng thời gian thực.

Phương pháp một giai đoạn, tiêu biểu là YOLO và SSD, thực hiện đồng thời việc định vị (localization) và phân loại (classification) chỉ trong một lần truyền qua mạng. Nhờ vậy, tốc độ xử lý nhanh hơn đáng kể trong khi vẫn giữ được độ chính xác cạnh tranh, rất thích hợp để triển khai trên các thiết bị có tài nguyên hạn chế.

#### **3.4.2 Các chỉ số đánh giá**

Để đánh giá khách quan chất lượng của một mô hình phát hiện đối tượng, cần sử dụng các chỉ số định lượng chuẩn.

IoU là tỷ lệ giữa diện tích phần giao và diện tích phần hợp của bounding box dự đoán và nhãn thực tế (bounding box ground truth). Hình 3.2 thể hiện công thức tính IoU nếu  $IoU = 0$  nghĩa là hai bounding box hoàn toàn không trùng nhau;  $IoU = 1$  nghĩa là hai bounding box khớp hoàn hảo. Một dự đoán được coi là đúng khi IoU với nhãn thực tế vượt qua một ngưỡng nhất định và tên lớp dự đoán đúng. Ngưỡng  $IoU = 0,50$  là tiêu chuẩn phổ biến nhất trong đánh giá phát hiện đối tượng [9][19].



Hình 3.2 Công thức tính IoU

Dựa trên IoU, mỗi dự đoán được phân loại vào một trong bốn trường hợp:

True Positive (TP) mô hình phát hiện đúng một đối tượng, vừa đúng vị trí vừa đúng lớp. Đây là trường hợp dự đoán chính xác mong muốn.

False Positive (FP) mô hình báo có đối tượng nhưng thực tế không có, hoặc khung bao lệch khỏi đối tượng thực hoặc nhận dạng sai lớp.



False Negative (FN) trong ảnh thực sự có đối tượng nhưng mô hình không phát hiện.

True Negative (TN) mô hình không đưa ra dự đoán tại những vùng vốn không có đối tượng. Trong bài toán phát hiện đối tượng, trường hợp này thường không được tính đến, vì số vùng nền không chứa đối tượng là gần như vô hạn; do đó các chỉ số đánh giá chủ yếu dựa trên TP, FP và FN.

Precision được tính theo công thức (3.4.1) là chỉ số đo lường mức độ đáng tin cậy của các dự đoán dương tính do mô hình đưa ra, phản ánh trong số tất cả các trường hợp mô hình dự đoán là có đối tượng thì bao nhiêu phần trăm thực sự đúng. Giá trị Precision càng cao cho thấy tỉ lệ báo nhầm càng thấp, nghĩa là khi mô hình dự đoán có đối tượng thì phần lớn dự đoán đó là chính xác.

$$Precision = \frac{TP}{TP + FP} \quad (3.4.1)$$

Recall được tính theo công thức (3.4.2) là chỉ số đo lường khả năng phát hiện đầy đủ của mô hình, phản ánh trong số tất cả các đối tượng thực sự có trong tập dữ liệu kiểm tra thì mô hình phát hiện được bao nhiêu phần trăm.

$$Recall = \frac{TP}{TP + FN} \quad (3.4.2)$$

F1-score là chỉ số kết hợp giữa Precision và Recall nhằm đánh giá hiệu năng tổng thể của mô hình một cách cân bằng. Được tính theo công thức (3.4.3), giá trị F1-score dao động trong khoảng từ 0 đến 1, giá trị càng cao cho thấy mô hình đạt được sự cân bằng càng tốt giữa khả năng phát hiện đầy đủ và độ chính xác của các dự đoán.

$$F1 = 2 \frac{Precision \cdot Recall}{Precision + Recall} \quad (3.4.3)$$

Average Precision (AP) là chỉ số đánh giá hiệu năng phát hiện đối tượng của mô hình trên từng lớp riêng lẻ. AP được tính bằng diện tích nằm dưới đường cong Precision-Recall (P-R curve) của một lớp diện tích càng lớn nghĩa là mô hình vừa phát hiện đầy đủ vừa ít báo nhầm đối với lớp đó.

Mean Average Precision at  $\text{IoU} = 0,50$  (mAP50) là độ chính xác trung bình của toàn mô hình Sau khi đã có AP của từng lớp, mAP50 được tính bằng cách lấy trung bình cộng AP của tất cả các lớp, trong đó một dự đoán được xem là đúng vị trí khi IoU với khung thực đạt từ 0,50 trở lên. Như vậy, mAP50 gộp kết quả của mọi lớp thành một con số duy nhất, đại diện cho hiệu năng chung của mô hình trên toàn bộ tập dữ liệu.

Ma trận nhầm lẫn (Confusion Matrix) là một bảng dùng để đánh giá hiệu năng của mô hình phân loại bằng cách đối chiếu giữa nhãn dự đoán của mô hình và nhãn thực tế của dữ liệu. Trong ma trận, các hàng biểu diễn nhãn thực tế và các cột biểu diễn nhãn được mô hình dự đoán, mỗi ô thể hiện số lượng mẫu tương ứng. Các giá trị nằm trên đường chéo chính là số mẫu được dự đoán đúng, trong khi các giá trị ngoài đường chéo cho biết mô hình đã nhầm lẫn lớp này thành lớp khác. Nhờ đó, ma trận nhầm lẫn giúp quan sát chi tiết mô hình hoạt động tốt ở lớp nào và thường nhầm lẫn giữa những lớp nào với nhau, từ đó xác định những điểm cần cải thiện về dữ liệu hoặc mô hình.

### **3.5 YOLO**

#### **3.5.1 Lịch sử phát triển**

YOLO được giới thiệu lần đầu bởi Redmon và cộng sự tại hội nghị CVPR 2016, với ý tưởng cốt lõi hoàn toàn khác biệt so với các phương pháp đương thời thực hiện toàn bộ quá trình phát hiện đối tượng trong một lần truyền qua mạng nơ-ron duy nhất [15]. Thay vì xử lý từng vùng đề xuất riêng lẻ như Faster R-CNN, YOLO chia ảnh thành lưới  $S \times S$  ô, mỗi ô dự đoán đồng thời nhiều bounding box và xác suất lớp. Phiên bản đầu tiên đạt tốc độ xử lý 45 khung hình/giây trên GPU nhanh hơn gần 100 lần so với Faster R-CNN trong khi vẫn duy trì độ chính xác cạnh tranh.

Qua nhiều thế hệ, dòng mô hình YOLO liên tục được cải tiến về cả kiến trúc lẫn phương pháp huấn luyện. YOLOv2 (2017) giới thiệu anchor box và batch normalization, cải thiện đáng kể độ chính xác phát hiện đối tượng nhỏ. YOLOv3 (2018) áp dụng phát hiện đa tỷ lệ ở ba kích thước khác nhau, tăng khả năng phát hiện vật thể nhỏ, vừa và lớn đồng thời. YOLOv4 (2020) tích hợp nhiều kỹ thuật tăng cường huấn luyện tiên tiến. YOLOv5 (2020) do Ultralytics phát triển, chuyển sang PyTorch và tạo ra hệ sinh thái công cụ thuận tiện. YOLOv8 (2023) tinh chỉnh toàn diện kiến trúc với đầu decoupled và anchor-free

design. Đến YOLOv11 (2024) tiếp tục cải tiến kiến trúc backbone và neck với hiệu suất cao hơn nhưng số tham số ít hơn. Gần đây nhất, YOLO26 (14 tháng 1 năm 2026) do Ultralytics phát hành tiếp tục cải tiến theo hướng tối ưu cho việc triển khai trên thiết bị biên, với nhiều thay đổi quan trọng trong kiến trúc và quy trình huấn luyện.

### **3.5.2 Giới thiệu về mô hình YOLO26**

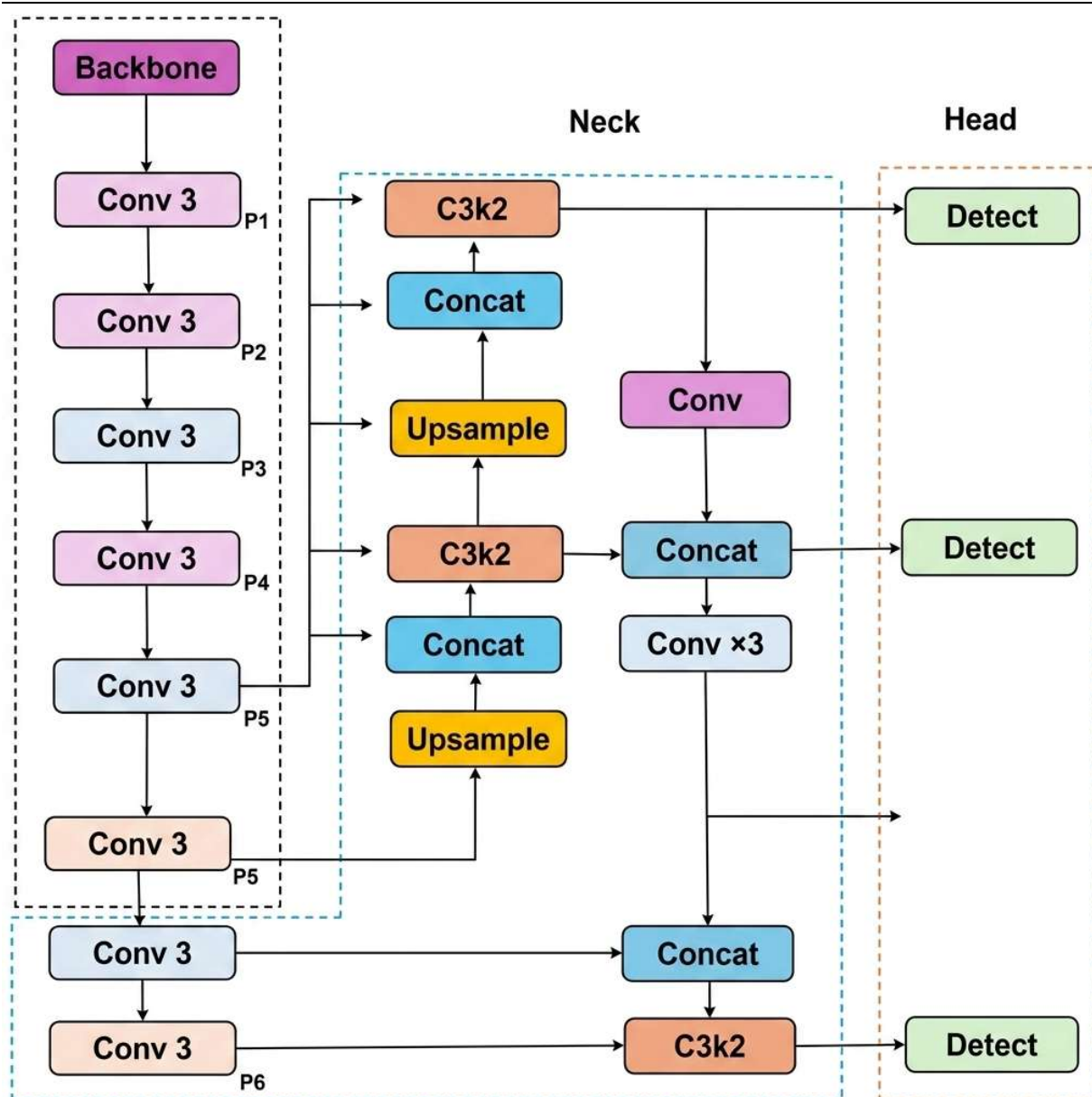
YOLO26 là họ mô hình thị giác máy tính thời gian thực hợp nhất do Ultralytics phát triển, ra mắt vào đầu năm 2026. Khác với việc chỉ phục vụ một tác vụ, YOLO26 được thiết kế như một mô hình thống nhất hỗ trợ nhiều tác vụ thị giác máy tính trong cùng một quy trình như phát hiện đối tượng, phân đoạn thực thể (instance segmentation), phân đoạn ngữ nghĩa (semantic segmentation), phân loại ảnh, ước lượng tư thế (pose estimation) và phát hiện đối tượng có hướng (oriented detection) [22].

Về tổng thể, YOLO26 vẫn kế thừa thiết kế của các phiên bản trước, đó là mô hình phát hiện đối tượng một giai đoạn (single-stage). Điểm nhấn của phiên bản này là khả năng suy luận đầu cuối (end-to-end), đầu phát hiện được làm gọn nhẹ và quy trình huấn luyện được cập nhật, hướng đến việc triển khai chính xác và hiệu quả trên thiết bị biên.

YOLO26 được cung cấp dưới năm biến thể có độ lớn khác nhau gồm YOLO26n, YOLO26s, YOLO26m, YOLO26l, YOLO26x nhằm cân đối giữa độ chính xác và tốc độ xử lý. Trên tập dữ liệu chuẩn COCO, các biến thể đạt mức mAP từ 40,9 đến 57,5 tương ứng với độ trễ suy luận từ 1,7 đến 11,8 ms trên phần cứng T4 TensorRT. Đặc biệt, biến thể nhỏ nhất YOLO26n đạt tốc độ suy luận CPU nhanh hơn tới 43% so với YOLO11n khi chạy trên bộ xử lý Intel Xeon @2,00GHz [22]. Do đó đề tài lựa chọn biến thể YOLO26n làm mô hình nhận diện cho đề tài, bởi sản phẩm cuối hướng đến việc chạy trên thiết bị di động của người dùng, vốn hạn chế về tài nguyên tính toán.

### **3.5.3 Kiến trúc YOLO26**

Hình 3.3 thể hiện kiến trúc YOLO26 gồm ba thành phần chính nối tiếp nhau Backbone, Neck và Head. Ảnh đầu vào là ảnh ba kênh màu, được xử lý lần lượt qua ba thành phần này để cho ra kết quả phát hiện.



Hình 3.3 Kiến trúc YOLO26

Backbone là mạng trích xuất đặc trưng. Thành phần này sử dụng các khối tích chập (Conv) để giảm dần độ phân giải của ảnh, kết hợp với các khối C3k2 để tạo ra các đặc trưng có mức độ trừu tượng tăng dần. Backbone xuất ra đặc trưng ở ba tỷ lệ không gian khác nhau và chuyển sang phần Neck, phục vụ cho chiến lược phát hiện đa tỷ lệ [13].

Neck là mạng tổng hợp đặc trưng, có nhiệm vụ kết hợp đặc trưng từ nhiều tỷ lệ trước khi đưa vào Head. Mở đầu phần Neck là khối SPPF (Spatial Pyramid Pooling Fast) giúp trích xuất đặc trưng ở nhiều mức độ trừu tượng; trong YOLO26 khối này được cải tiến bằng

một kết nối tắt (shortcut) để cải thiện luồng thông tin. Tiếp theo là khối C2PSA sử dụng cơ chế tự chú ý (self-attention) nhằm tăng khả năng mô hình hóa ngữ cảnh toàn cục. Phần Neck còn sử dụng các phép tăng độ phân giải (Upsample) và ghép nối (Concat) để hợp nhất đặc trưng giữa các tỷ lệ; ở giai đoạn cuối có bổ sung một khối chú ý (attention block) nhằm tăng cường đặc trưng [13].

Head là mạng phát hiện, YOLO26 sử dụng ba đầu phát hiện hoạt động trên ba tỷ lệ đặc trưng khác nhau, lần lượt chuyên cho việc phát hiện các đối tượng có kích thước nhỏ, trung bình và lớn [13]. Cần lưu ý rằng kích thước đối tượng ở đây mang tính tương đối, được xét theo tỷ lệ so với kích thước của ảnh hoặc khung hình đầu vào.

#### **3.5.4 Các cải tiến chính của YOLO26**

So với các phiên bản trước, YOLO26 tập trung vào việc đơn giản hóa quá trình suy luận, ổn định huấn luyện và nâng cao hiệu quả triển khai. Các cải tiến chính được Ultralytics công bố như sau:

Hồi quy không dùng DFL (DFL-Free Regression): YOLO26 loại bỏ DFL khỏi đầu phát hiện. Việc này giúp giảm độ phức tạp của đầu phát hiện và đơn giản hóa quá trình xuất mô hình, trong khi vẫn duy trì khoảng hồi quy không bị giới hạn [22].

Suy luận đầu cuối không cần NMS: Khác với các mô hình phát hiện truyền thống vốn cần bước hậu xử lý Non-Maximum Suppression (NMS) riêng biệt, YOLO26 mặc định hoạt động theo cơ chế đầu cuối (end-to-end) kết quả được sinh trực tiếp mà không cần NMS, qua đó giảm độ trễ và đơn giản hóa việc tích hợp vào sản phẩm .

Sử dụng kiến trúc hai đầu phát hiện: Thực chất, YOLO26 sử dụng kiến trúc hai đầu phát hiện (dual-head). Đầu một-một (one-to-one) là đầu mặc định, cho ra kết quả đầu cuối không cần NMS với tối đa 300 phát hiện trên mỗi ảnh, được tối ưu cho tốc độ suy luận và triển khai đơn giản. Đầu một-nhiều (one-to-many) cho ra kết quả theo kiểu YOLO truyền thống và cần đến NMS đầu này thường đạt độ chính xác nhỉnh hơn một chút nhưng tốn thêm bước xử lý. Người dùng có thể lựa chọn giữa hai đầu này khi suy luận, kiểm định hoặc xuất mô hình, tùy theo ưu tiên về tốc độ hay độ chính xác.

## ***Xây dựng ứng dụng nhận diện bộ cánh cứng***

---

Cân bằng hàm mất mát lũy tiến (Progressive Loss): YOLO26 áp dụng kỹ thuật Progressive Loss để dịch chuyển dần trọng tâm huấn luyện về phía đầu phát hiện được dùng khi suy luận. Nhờ đó, hành vi trong quá trình huấn luyện và quá trình suy luận trở nên nhất quán hơn, giúp việc tối ưu ổn định hơn.

Gán nhãn nhận biết đối tượng nhỏ: YOLO26 giới thiệu phương pháp gán nhãn STAL (Small-Target-Aware Label Assignment) nhằm cải thiện việc bao phủ nhãn dương cho các đối tượng có kích thước nhỏ. Phương pháp này giúp hạn chế tình trạng bỏ sót đối tượng nhỏ trong quá trình huấn luyện, qua đó nâng cao khả năng phát hiện các đối tượng nhỏ.

Trình tối ưu MuSGD: YOLO26 sử dụng một trình tối ưu lai (hybrid optimizer) có tên MuSGD mang các ý tưởng tối ưu từ lĩnh vực mô hình ngôn ngữ lớn sang bài toán thị giác máy tính, giúp quá trình huấn luyện ổn định hơn.

Hiệu quả triển khai: Nhờ đầu phát hiện được làm gọn nhẹ và đường suy luận mặc định không cần NMS YOLO26 giảm đáng kể chi phí suy luận trên nhiều nền tảng phần cứng và định dạng xuất mô hình khác nhau. Như đã nêu, biến thể YOLO26n đạt tốc độ suy luận rất tốt. YOLO26 hỗ trợ nhiều định dạng xuất phổ biến như TensorRT, ONNX, CoreML, TFLite và OpenVINO, thuận tiện cho việc triển khai trên các thiết bị khác nhau.

### **3.6 Roboflow**

#### **3.6.1 Giới thiệu về Roboflow**

Roboflow là nền tảng phần mềm toàn diện dành cho các nhà phát triển ứng dụng thị giác máy tính, cung cấp đầy đủ công cụ cho toàn bộ quy trình từ quản lý dữ liệu, gán nhãn, tiền xử lý, tăng cường, đến huấn luyện và triển khai mô hình. Được thành lập năm 2019, Roboflow nhanh chóng trở thành một trong những nền tảng phổ biến nhất trong cộng đồng nghiên cứu thị giác máy tính nhờ giao diện trực quan và khả năng tích hợp liền mạch với các framework học sâu phổ biến như Ultralytics YOLO, TensorFlow và PyTorch.

Roboflow Universe là nhánh cộng đồng của nền tảng, nơi các nhà nghiên cứu và tổ chức trên toàn thế giới chia sẻ công khai hơn 200.000 tập dữ liệu thị giác máy tính được gán nhãn sẵn, bao phủ hàng trăm lĩnh vực ứng dụng khác nhau. Đây là nguồn dữ liệu quan trọng được sử dụng trong đề tài để tải về các dataset bộ cánh cứng đã được gán nhãn theo định dạng YOLO.

---

### **3.6.2 Tính năng gán nhãn và kho dữ liệu cộng đồng**

Roboflow cung cấp công cụ gán nhãn trực tuyến Roboflow Annotate, hỗ trợ nhiều dạng chú thích khác nhau tùy theo loại bài toán bounding box phục vụ phát hiện đối tượng, đa giác (polygon) và mặt nạ điểm ảnh (mask) phục vụ phân đoạn, nhãn lớp phục vụ phân loại ảnh, cũng như điểm khóa (keypoint) phục vụ ước lượng tư thế. Đối với bài toán phát hiện đối tượng như trong đề tài sử dụng công cụ bounding box để vẽ hộp chữ nhật bao quanh đối tượng rồi gán nhãn lớp tương ứng, hộp bao cần được vẽ ôm sát đối tượng để mô hình có thể học được đặc trưng một cách chính xác hơn.

Roboflow Annotate còn tích hợp các tính năng gán nhãn có hỗ trợ của trí tuệ nhân tạo nhằm tăng tốc quá trình gán nhãn. Tiêu biểu là Label Assist, cho phép tự động gợi ý khung bao dựa trên một mô hình công khai hoặc các mô hình đã huấn luyện sẵn trên Roboflow Universe và Smart Polygon, cho phép tạo nhanh chú thích đa giác chỉ bằng thao tác nhấp chuột vào đối tượng. Tính năng nổi bật nhất là Label Assist tích hợp Segment Anything Model (SAM) của Meta AI cho phép gán nhãn bán tự động chỉ bằng một lần click vào đối tượng, SAM sẽ tự động phát hiện và khoanh vùng đối tượng đó với độ chính xác cao, giúp tăng tốc quá trình annotation lên nhiều lần so với vẽ thủ công hoàn toàn. Cho phép xuất dữ liệu ra nhiều định dạng phổ biến như YOLO, COCO JSON hay Pascal VOC, thuận tiện cho việc huấn luyện trên các nền tảng khác nhau.

Bên cạnh công cụ gán nhãn, Roboflow còn phát triển Roboflow Universe một kho dữ liệu thị giác máy tính cộng đồng theo hướng mã nguồn mở, được giới thiệu từ năm 2021 và hiện là một trong những kho dữ liệu lớn nhất trong lĩnh vực này. Theo tài liệu chính thức của Roboflow, kho dữ liệu này cung cấp hơn 200.000 bộ dữ liệu và hơn 50.000 mô hình đã được huấn luyện sẵn. [16].

Nhờ kho dữ liệu cộng đồng này, có thể bổ sung và mở rộng dữ liệu cho đề tài mà không phải thu thập và gán nhãn hoàn toàn từ đầu, qua đó tiết kiệm đáng kể thời gian ở giai đoạn chuẩn bị dữ liệu vốn là một trong những công đoạn tốn nhiều công sức nhất trong quá trình xây dựng mô hình thị giác máy tính.

### **3.6.3 Tính năng tiền xử lý và tăng cường dữ liệu**

Roboflow cung cấp một bộ công cụ tiền xử lý mạnh mẽ giúp chuẩn hóa dữ liệu trước khi đưa vào huấn luyện. Về resize ảnh, Roboflow cho phép chuẩn hóa toàn bộ ảnh về một kích thước thống nhất với nhiều chế độ như stretch (kéo giãn), fit (giữ tỉ lệ với padding) hoặc crop (cắt trung tâm). Hỗ trợ tự động căn chỉnh hướng ảnh (auto-orient) dựa trên metadata EXIF của ảnh chụp từ điện thoại, tránh tình trạng ảnh bị lật ngược.

Về tăng cường dữ liệu (data augmentation), Roboflow cung cấp giao diện trực quan để cấu hình các phép biến đổi ngẫu nhiên áp dụng lên tập huấn luyện, bao gồm: lật ngang và dọc (flip), xoay ảnh (rotation) trong khoảng góc tùy chọn, cắt trung tâm (crop), thay đổi độ sáng (brightness) và tương phản (contrast), thêm nhiễu (noise), làm mờ (blur) và thay đổi màu sắc (hue/saturation). Một lợi thế quan trọng của việc thực hiện augmentation trên Roboflow là các nhãn bounding box được tự động cập nhật tương ứng với mỗi phép biến đổi hình học, đảm bảo sự chính xác kể cả khi hình ảnh đã thay đổi.

Tính năng Dataset Health Check của Roboflow tự động phân tích toàn bộ dataset sau khi tải lên và báo cáo các vấn đề tiềm ẩn như ảnh trùng lặp (duplicate images), lớp mất cân bằng nghiêm trọng (class imbalance), bounding box có kích thước bất, annotation null (ảnh không có nhãn) và nhiều vấn đề khác. Công cụ này giúp phát hiện và xử lý sớm các lỗi dữ liệu trước khi đưa vào huấn luyện, tiết kiệm thời gian và tránh ảnh hưởng tiêu cực đến chất lượng mô hình.

## **3.7 RESTful API**

### **3.7.1 Giới thiệu về REST**

REST là một phong cách kiến trúc phần mềm dành cho các hệ thống phân tán, được Roy Fielding trình bày trong luận án tiến sĩ năm 2000 tại Đại học California, Irvine [7]. REST được đề xuất nhằm xây dựng các dịch vụ mạng có khả năng mở rộng, đáng tin cậy và hiệu quả, và đã trở thành một trong những nền tảng quan trọng cho việc thiết kế các giao diện lập trình ứng dụng trên nền web.

### **3.7.2 Các ràng buộc kiến trúc**

REST định nghĩa một tập các ràng buộc kiến trúc mà một hệ thống cần tuân theo, trong đó các ràng buộc tiêu biểu gồm mô hình client-server tách biệt giao diện người dùng

---



khởi phần lưu trữ và xử lý dữ liệu; tính phi trạng thái (stateless), theo đó mỗi yêu cầu từ phía client phải chứa đầy đủ thông tin cần thiết và máy chủ không lưu trạng thái của các yêu cầu trước đó; và giao diện đồng nhất (uniform interface) giúp đơn giản hóa và đồng bộ cách thức tương tác giữa các thành phần [7].

### **3.7.3 RESTful API**

Một giao diện lập trình ứng dụng tuân theo các ràng buộc của REST được gọi là RESTful API. Trên nền web, RESTful API thường sử dụng các phương thức của giao thức HTTP để thao tác với tài nguyên và trao đổi dữ liệu, phổ biến nhất là định dạng JSON. RESTful API đóng vai trò trung gian tiếp nhận ảnh được gửi từ ứng dụng di động, chuyển đến mô hình nhận diện để xử lý và trả về kết quả dưới dạng JSON.

## **3.8 Flask**

### **3.8.1 Giới thiệu về Flask**

Flask là một micro web framework viết bằng Python, được tạo ra bởi Armin Ronacher và ra mắt lần đầu năm 2010. Flask được xây dựng trên nền tảng hai thư viện cốt lõi là Werkzeug và Jinja2. Flask chỉ cung cấp những thành phần thực sự cần thiết cho một web framework định tuyến URL, xử lý request-response và quản lý context, để lại tất cả các quyết định thiết kế còn lại cho nhà phát triển tự lựa chọn.

### **3.8.2 Các tính năng**

Flask cung cấp hệ thống định tuyến linh hoạt thông qua decorator `@app.route()` cho phép ánh xạ bất kỳ URL pattern nào đến hàm xử lý tương ứng. Flask hỗ trợ đầy đủ các HTTP method và cho phép xác định method nào được chấp nhận tại từng route. Đối tượng request của Flask cung cấp truy cập dễ dàng đến tất cả dữ liệu của request để form data, JSON body, file upload, query string và header. Hàm `jsonify()` của Flask cho phép chuyển đổi dict Python thành JSON

Hệ sinh thái extension phong phú của Flask cho phép mở rộng chức năng một cách có kiểm soát. Flask-CORS giải quyết vấn đề same-origin policy khi ứng dụng Flutter gọi API từ một domain khác, đảm bảo cross-platform request hoạt động bình thường. Flask-Limiter giúp giới hạn số lượng request mỗi client có thể gửi trong một khoảng thời gian bảo vệ server khỏi bị quá tải.

### **3.9 Phát triển ứng dụng di động với Flutter**

#### **3.9.1 Giới thiệu Flutter**

Flutter là bộ công cụ phát triển giao diện người dùng mã nguồn mở do Google tạo ra, ra mắt phiên bản beta năm 2018 và đạt phiên bản ổn định 1.0 vào tháng 12 năm 2018. Flutter cho phép xây dựng ứng dụng đa nền tảng cho Android, iOS, web, Windows, macOS và Linux từ một codebase Dart duy nhất. Tính đến năm 2026, Flutter đã được sử dụng bởi hơn 1 triệu nhà phát triển tích cực hàng tháng và là framework phát triển ứng dụng di động đa nền tảng phổ biến nhất theo nhiều khảo sát của cộng đồng lập trình viên [11].

Điểm khác biệt kiến trúc quan trọng nhất của Flutter so với các framework đa nền tảng khác như React Native hay Xamarin là Flutter không dùng bridge để giao tiếp với các thành phần UI gốc của hệ điều hành thay vào đó Flutter có engine rendering riêng và tự vẽ toàn bộ giao diện lên canvas trực tiếp. Điều này đảm bảo giao diện hoàn toàn nhất quán trên mọi nền tảng giúp Flutter đạt hiệu năng rendering 60fps hoặc 120fps trên phần cứng.

#### **3.9.2 Ngôn ngữ Dart**

Flutter sử dụng ngôn ngữ lập trình Dart ngôn ngữ hướng đối tượng (OOP) do Google phát triển, được thiết kế đặc biệt để xây dựng giao diện người dùng hiệu suất cao. Dart có cú pháp gần gũi với Java, C# và JavaScript, nên lập trình viên có nền tảng OOP thường học Dart khá nhanh. Ngôn ngữ Dart được thiết kế với mục tiêu null-safety từ phiên bản Dart 2.12 nghĩa là compiler bắt buộc lập trình viên phải xử lý tường minh các trường hợp null, giảm đáng kể lỗi NullPointerException phổ biến trong lập trình truyền thống.

Dart hỗ trợ hai chế độ biên dịch tùy theo môi trường biên dịch AOT trong ứng dụng production, chuyển code Dart thành mã máy ARM hoặc Intel x64 native giúp ứng dụng Flutter chạy nhanh và biên dịch JIT trong môi trường phát triển, cho phép tính năng Hot Reload thay đổi code và xem kết quả tức thì mà không cần khởi động lại ứng dụng. Dart hỗ trợ lập trình bất đồng bộ thông qua cú pháp async/await và kiểu Future/Stream, giúp xử lý các tác vụ I/O như gọi HTTP API, đọc ghi file mà không chặn luồng UI chính, đảm bảo giao diện luôn mượt mà trong khi chờ phản hồi từ server.

### **3.9.3 Ưu nhược điểm của Flutter**

Về khả năng đa nền tảng, một codebase Flutter duy nhất tạo ra ứng dụng chạy tốt trên cả Android và iOS. Về tốc độ phát triển, tính năng Hot Reload cho phép nhìn thấy thay đổi giao diện gần như ngay lập tức giúp tăng tốc đáng kể vòng lặp thiết kế và gỡ lỗi. Về giao diện người dùng, Flutter cung cấp thư viện widget phong phú tuân theo chuẩn Material Design 3 cho Android và Cupertino cho iOS, giúp xây dựng giao diện đẹp và chuyên nghiệp mà không cần kiến thức thiết kế sâu. Về hiệu năng, nhờ biên dịch AOT và engine rendering tự quản lý, ứng dụng Flutter đạt hiệu năng cao, hoàn toàn đủ cho các tác vụ gọi API và hiển thị kết quả nhận diện trong đề tài này.

Bên cạnh các ưu điểm, Flutter cũng có một số hạn chế cần lưu ý. Kích thước file APK/IPA của ứng dụng Flutter thường lớn do phải đóng gói kèm Flutter engine. Một số tính năng phần cứng đặc thù của từng nền tảng có thể yêu cầu viết platform channel bằng Kotlin để gọi API native đòi hỏi kiến thức về phát triển native.

## **Chương 4: MÔ HÌNH NHẬN DẠNG BỘ CÁNH CỨNG**

Chương này tập trung vào quá trình xây dựng mô hình nhận diện bộ cánh cứng trên thực tế. Giới thiệu về các loại bộ cánh cứng được chọn làm dữ liệu nhận diện cho đề tài cùng với đó là trình bày lần lượt các bước trong quy trình xây dựng mô hình.

### **4.1 Giới thiệu bộ cánh cứng**

Bộ cánh cứng (Coleoptera) là bộ lớn nhất trong lớp côn trùng. Côn trùng thuộc bộ cánh cứng có kích thước khác nhau rõ rệt từ rất nhỏ hơn 1mm đến rất lớn trên 75mm. Bộ này phân bố rất rộng rãi, đặc biệt chúng luôn hiện diện ở những cánh rừng rậm với hệ sinh thái đa dạng và những nơi có nguồn thức ăn dồi dào.

Phần lớn các loài côn trùng bộ Cánh cứng có hai đôi cánh. Đôi cánh trước có cấu tạo bằng chất sừng cứng, đôi cánh sau bằng chất màng, thường dài hơn đôi cánh trước và khi ở trạng thái nghỉ đôi cánh sau thường xếp lại dưới đôi cánh trước. Miệng của các loài côn trùng thuộc bộ này có kiểu gặm nhai, hai hàm rất phát triển. Côn trùng thuộc bộ Cánh cứng thuộc nhóm biến thái hoàn toàn. Sâu non có nhiều hình dạng khác nhau, nhưng đa số có dạng chân chạy hoặc dạng bộ hung. Nhộng đa số là nhộng trần, một số loài như xén tóc nhộng được bao phủ bởi một lớp kén mỏng. Côn trùng thuộc bộ Cánh cứng thường đẻ trứng trong đất, trong vỏ thân cây, trong mô lá, trong nước.

Nhiều loài bộ cánh cứng là dịch hại nguy hiểm, gây thiệt hại nghiêm trọng cho cây trồng có giá trị kinh tế cao như cây dừa, cây ăn trái, hoa màu và rau. Đáng chú ý, ở nhiều loài, cả hai giai đoạn ấu trùng và trưởng thành đều có khả năng gây hại trên các bộ phận khác nhau của cây khiến việc nhận diện sớm và chính xác từng loài trở thành yêu cầu quan trọng trong công tác quản lý dịch hại.

### **4.2 Các loại bộ cánh cứng được thu thập trong đề tài**

Đề tài tập trung vào 9 loài phổ biến và có tác hại kinh tế cao tại khu vực đồng bằng sông Cửu Long. Đây cũng là các loại có nhiều hình ảnh trên các nền tảng trực tuyến.

#### **4.2.1 Bộ dừa**

Bộ dừa hình 4.1, còn gọi là bộ cánh cứng hại dừa, có tên khoa học *Brontispa longissima* (Gestro). Thành trùng có thân thuôn dài khoảng 8-12 mm, màu nâu đỏ đến nâu đen bóng, cánh cứng mang các đường gân dọc rõ, đầu nhỏ và râu tương đối dài. Vòng đời gồm bốn giai đoạn trứng → ấu trùng → nhộng → trưởng thành; trứng có màu nâu sậm, hình hơi bầu dục, được đẻ riêng lẻ trên lá non chưa bung. Cả ấu trùng và thành trùng đều thích sống trong các tàu lá non còn xếp lại ở phần đọt. Loài này có nguồn gốc từ vùng Đông Nam Á -Thái Bình Dương; tại Việt Nam, bộ dừa xuất hiện và bùng phát thành dịch từ cuối năm 1999, sau đó lan rộng nhanh chóng khắp các tỉnh phía Nam [3].



Hình 4.1 Bộ dừa

Ký chủ chính của bộ dừa là cây dừa và các loài cây họ cau. Cả ấu trùng và thành trùng đều tập trung ở mặt trên của lá non, tạo ra những vết nâu chạy dọc theo gân lá như hình 4.2. Khi mật độ cao, các vết này liên kết lại làm lá khô cháy, quăn queo như bị lửa tấp. Cây bị hại nặng mất khả năng quang hợp, sinh trưởng kém, giảm năng suất rõ rệt và có thể suy kiệt dần. Bộ dừa được xem là một trong những đối tượng gây hại nguy hiểm nhất trên cây dừa và các loại cây cùng họ.



Hình 4.2 Tác hại của bộ dừa

Biện pháp phòng chống: Biện pháp sinh học phóng thích và bảo vệ thiên địch, đặc biệt là ong ký sinh *Asecodes hispinarum* và bọ đuôi kìm; sử dụng chế phẩm nấm ký sinh côn trùng nấm xanh *Metarhizium*, nấm trắng *Beauveria* phun lên đọt dừa. Biện pháp canh tác vệ sinh vườn, cắt tỉa và tiêu hủy các tàu lá bị hại nặng nhằm giảm nguồn dịch. Biện pháp hóa học khi mật độ cao, dùng thuốc bảo vệ thực vật có tính lưu dẫn hoặc xông hơi, phun hoặc đặt vào nách lá non.

#### **4.2.2 Bọ vòi voi hại dừa**

Bọ vòi voi hại dừa hình 4.3 có tên khoa học *Diocalandra frumenti* (Fabricius). Thành trùng màu nâu đen, thân dài khoảng 7-8 mm, rộng khoảng 1,5 mm, trên cánh trước có các đốm vàng đặc trưng ở đầu cánh và cuối cánh, đầu kéo dài thành vòi. Thành trùng sợ ánh sáng, hoạt động mạnh vào lúc chiều tối và thường ẩn ở chỗ tiếp xúc giữa hai trái hoặc gần cuống trái. Trứng được đẻ trên vỏ trái gần cuống hoặc bên trong đường hầm có sẵn; ấu trùng màu vàng nhạt, đục thành đường hầm trong vỏ trái. Vòng đời kéo dài khoảng 2-3 tháng, trải qua bốn giai đoạn: trứng → ấu trùng → nhộng → trưởng thành. Đây là một trong những loài gây hại phổ biến trên dừa tại các tỉnh Đồng bằng sông Cửu Long [3].



Hình 4.3 Bọ vòi voi hại dừa

Bọ vòi voi gây hại trên nhiều bộ phận của cây dừa gồm rễ, thân và đặc biệt là trái. Ấu trùng đục đường hầm trong vỏ trái gần cuống như hình 4.4, làm trái chậm phát triển, rụng non hoặc giảm chất lượng; mỗi buồng trái bị hại thường có một vài thành trùng cư trú. Khi tấn công cuống lá và bẹ lá, vết đục làm lá biến vàng và dễ đổ gãy, biểu hiện bắt đầu từ các lá phía ngoài rồi lan dần vào trong, ảnh hưởng đến sinh trưởng và năng suất của cây.



Hình 4.4 Tác hại của bọ vòi voi

**Biện pháp phòng chống:** Biện pháp sinh học bảo vệ và lợi dụng thiên địch như kiến vàng (*Oecophylla smaragdina*), ong ký sinh (*Spathius apicalis*) và các loài bọ ăn thịt, sử dụng chế phẩm nấm xanh *Metarhizium anisopliae* để giảm mật số quần thể. Biện pháp canh tác vệ sinh vườn dừa thông thoáng, thu gom và tiêu hủy trái rụng, bẹ lá khô để hạn chế nơi trú ẩn và đẻ trứng; bón phân, tưới tiêu hợp lý giúp cây khỏe, tăng sức chống chịu. Biện pháp hóa học khi mật độ cao, dùng thuốc bảo vệ thực vật phù hợp rải gốc hoặc phun lên buồng, cuống trái vào thời điểm thành trùng hoạt động.

#### **4.2.3 Đuông dừa**

Đuông dừa hình 4.5 có tên khoa học *Rhynchophorus ferrugineus* (Olivier). Thành trùng là bộ cánh cứng kích thước lớn, dài khoảng 2-4 cm, màu nâu đỏ, có vòi dài đặc trưng. Trứng màu trắng sữa, nở sau 3-4 ngày. Ấu trùng màu trắng sữa hoặc vàng nhạt, thân mập tròn không chân, dài 40-50 mm, gồm khoảng 13 đốt, miệng cứng rất phát triển, cuối đuôi đẹp. Nhộng ban đầu màu trắng sữa rồi chuyển nâu, dài khoảng 35mm[3].



Hình 4.5 Đuông dừa

Ký chủ chính là cây dừa và các cây họ Cau. Đuông dừa thường là côn trùng xâm nhập thứ cấp thành trùng tìm đến đẻ trứng ở những vết thương, vết nứt trên thân hoặc đọt dừa, đặc biệt là các vết do kiến vương tạo ra. Ấu trùng nở ra đục khoét sâu vào thân và đỉnh sinh trưởng, ăn rỗng phần mô mềm bên trong. Triệu chứng ban đầu là vài lỗ nhỏ trên thân hoặc đọt, có mùn gỗ và ít nhựa màu nâu chảy ra sau đó mô bên trong lên men, bốc mùi hôi, lá khô và rụng dần từ đọt xuống, cuối cùng cây có thể chết như hình 4.6. Đây là dịch hại đặc biệt nguy hiểm, có khả năng gây chết cây hàng loạt, nhất là trên vườn dừa tơ.

Biện pháp phòng chống: Biện pháp sinh học phòng trừ kiến vương triệt để vì đây là tác nhân tạo nơi cho đuông xâm nhập; tránh gây tổn thương cơ giới cho thân dừa và xử lý kịp thời các vết thương, vết nứt; sử dụng bẫy pheromone dẫn dụ thành trùng. Biện pháp canh tác vệ sinh vườn, dọn bỏ xác cây dừa chết, gốc mục nơi đuông thường đẻ trứng. Biện pháp hóa học khi phát hiện cây bị hại có thể khoan lỗ tại điểm bị tấn công, bơm thuốc trừ sâu vào rồi bịt kín bằng đất sét.





Hình 4.6 Tác hại của đung dừa

#### **4.2.4 Kiến vương**

Kiến vương hình 4.7 hay bộ hung tê giác một sừng có tên khoa học *Oryctes rhinoceros* (Linnaeus). Thành trùng có thân cứng chắc, màu nâu đen bóng, dài khoảng 35-50 mm. Con đực mang một sừng cong đặc trưng trên đầu, sừng con đực dài hơn con cái. Ấu trùng hình chữ C, màu trắng đục, đầu màu nâu, thường sống trong các đồng phân chuồng, rác mục và xác thực vật đang phân hủy. Vòng đời gồm bốn giai đoạn trứng → ấu trùng → nhộng → thành trùng. Kiến vương hiện diện hầu như quanh năm tại các vùng trồng dừa Đồng bằng sông Cửu Long, với một số đợt cao điểm gây hại trong năm[3].



Hình 4.7 Kiến vương

Giai đoạn gây hại chính là thành trùng. Kiến vương đục vào phần mô mềm ở cuối bẹ lá, bó lá ngọn và đỉnh sinh trưởng của cây dừa để ăn các lá non đang phát triển. Khi lá

### ***Xây dựng ứng dụng nhận diện bọ cánh cứng***

---

bung ra sẽ mang hình tam giác hoặc bị cắt hình răng lược rất đặc trưng; chúng còn cắn phá bông mo và hoa dừa làm rụng hoa. Cây bị tấn công liên tục sẽ mất sức, kém phát triển, trường hợp nặng có thể chết. Nguy hiểm hơn, vết thương do kiến vương tạo ra chính là điều kiện thuận lợi để đuông dừa và nấm bệnh xâm nhập, gây hại thứ cấp.

Biện pháp phòng chống: Biện pháp sinh học bảo vệ thiên địch như kiến vàng; sử dụng bẫy pheromone tập hợp để giảm mật số quần thể. Biện pháp canh tác vệ sinh vườn, dọn sạch và xử lý các đồng rác mục, phân chuồng, gốc dừa mục nơi kiến vương đẻ trứng và ấu trùng phát triển; bắt thủ công thành trùng trong các nách lá, dùng móc sắt kéo bắt khi phát hiện hang. Biện pháp hóa học rải thuốc hạt vào các nách lá non vào đầu mùa mưa để phòng ngừa; bỏ thuốc hạt vào hang rồi bịt kín khi cây đã bị tấn công.

#### **4.2.5 Bọ hà khoai lang**

Bọ hà hình 4.8 còn gọi là sùng khoai lang có tên khoa học *Cylas formicarius* (Fabricius). Thành trùng dài khoảng 5-8 mm, thân thuôn dài trông gần giống con kiến; phần đầu và cánh cứng màu xanh đen bóng ánh kim, phần ngực và chân màu nâu đỏ. Trứng hình bầu dục, dài khoảng 0,7 mm, được đẻ rải rác trong các lỗ nhỏ trên củ hoặc thân gần mặt đất. Ấu trùng màu trắng ngà, không chân, dài khoảng 7-10 mm, đục thành đường hầm ngoằn ngoèo bên trong củ. Vòng đời hoàn chỉnh kéo dài khoảng 33 ngày; điều kiện khô nóng, đất nứt nẻ rất thuận lợi cho bọ hà phát triển [4].



Hình 4.8 Bọ hà khoai lang

### ***Xây dựng ứng dụng nhận diện bộ cánh cứng***

---

Ký chủ chính là cây khoai lang bộ hà được xem là dịch hại quan trọng nhất trên cây trồng này. Ấu trùng đục rỗng ruột củ tạo các đường hầm, kích thích củ tiết độc tố tự vệ khiến củ có vị đắng và mùi khó chịu, mất hoàn toàn giá trị thương phẩm. Thành trùng còn ăn biểu bì thân, lá và phá rễ làm cây còi cọc, lá vàng úa. Ở những vùng đất khô hạn, bộ hà có thể gây thất thu 40-50% sản lượng.

Biện pháp phòng chống: Biện pháp sinh học sử dụng bầy pheromone giới tính để dẫn dụ và tiêu diệt thành trùng đực; bảo vệ thiên địch tự nhiên như kiến, nhện, bọ rùa. Biện pháp canh tác luân canh cây trồng, đặc biệt trồng khoai sau vụ lúa nước để cắt đứt vòng đời và hạn chế tỷ lệ củ bị hại; chọn hom giống sạch, vun gốc kín để tránh củ lộ ra mặt đất và bị đẻ trứng. Biện pháp hóa học thu gom, tiêu hủy dẫy và củ bị hại sau thu hoạch; sử dụng thuốc bảo vệ thực vật khi cần thiết.

#### **4.2.6 Bọ rầy / Bù rầy**

Bọ rầy còn gọi là bù rầy, đuông đất, sùng trắng hay sùng đất có tên khoa học *Holotrichia sauteri* (Moser). Thành trùng là bọ cánh cứng màu nâu, hoạt động mạnh vào các tháng mùa mưa. Con cái dùng chân đào xới đất gần gốc cây rồi đẻ khoảng 15-17 trứng; trứng hình tròn, kích thước 2-3 mm. Ấu trùng chính là sùng đất có thân màu trắng ngà đến trắng xanh hoặc vàng, sống trong đất. Giai đoạn ấu trùng kéo dài rất lâu, gần một năm mới hóa nhộng. Bọ rầy thường sinh sống và sinh sản ở bãi bồi ven sông, vùng đất pha cát nhiều.



Hình 4.9 Bọ rầy

### ***Xây dựng ứng dụng nhận diện bộ cánh cứng***

---

Giai đoạn gây hại nghiêm trọng nhất là ấu trùng. Chúng đào hang dưới gốc và gặm phá bộ rễ của nhiều loại cây trồng như mía, bắp, gừng, khoai lang, bầu bí và hoa màu, làm cây giảm khả năng hút nước và dinh dưỡng, còi cọc, héo và chết. Do ấu trùng sống sâu trong đất sát vùng rễ và có vòng đời kéo dài, đây là đối tượng khó phòng trừ hơn nhiều so với kiến vương hay đuông dừa, có thể gây hại liên tục qua nhiều vụ.

Biện pháp phòng chống: Biện pháp sinh học dùng bẫy đèn bắt thành trùng vào các tháng mùa mưa nhằm hạn chế sinh sản; thu gom, tiêu diệt ấu trùng khi làm đất. Biện pháp canh tác cày bừa, phơi ải đất kỹ trước khi trồng để diệt trứng và ấu trùng; bón phân hữu cơ đã hoai mục hoàn toàn, tránh dùng phân chuồng tươi làm nơi đẻ trứng; luân canh với cây trồng nước. Biện pháp hóa học trộn thuốc bảo vệ thực vật dạng hạt vào đất khi làm đất để tiêu diệt ấu trùng mới nở.

#### **4.2.7 Câu cầu xanh**

Câu cầu xanh có tên khoa học *Hypomeces squamosus* (Fabricius). Thành trùng có cơ thể hình bầu dục, dài khoảng 10-15 mm, toàn thân phủ một lớp vảy bột màu vàng tươi óng ánh khi mới vũ hóa, sau chuyển dần sang xanh nhạt hoặc xám trắng ánh kim; mắt lồi, miệng dạng vòi khỏe. Câu cầu xanh thường sống thành từng cụm 3-4 con, ẩn dưới mặt lá và có tập tính giả chết. Trứng màu trắng ngà, hình bầu dục dài khoảng 1 mm, dễ rải rác trên mặt đất quanh gốc cây. Ấu trùng màu vàng nhạt, dài 15-20 mm, không chân, sống trong đất. Loài này phát sinh và gây hại quanh năm [4].



Hình 4.10 Câu cầu xanh

### ***Xây dựng ứng dụng nhận diện bộ cánh cứng***

---

Câu cầu xanh là loài côn trùng ăn tạp, gây hại trên nhiều loại cây như xoài, cam, quýt, bưởi, ổi, tiêu, sầu riêng và cao su. Thành trùng gặm khuyết lá non, lộc non, đọt và cả hoa, quả non, làm cây sinh trưởng và phát triển kém. Ấu trùng sống trong đất, cắn phá rễ và gốc cây, làm cây suy yếu từ dưới lên. Do thường bị xem nhẹ trong công tác phòng trừ, loài này dễ gây thiệt hại tích lũy đáng kể qua thời gian.

Biện pháp phòng chống: Biện pháp sinh học lợi dụng tập tính giả chết, rung cây hoặc quơ mạnh trên tán vào sáng sớm để thu gom thành trùng rơi xuống, kết hợp bắt tay khi cây ra đọt non. Biện pháp canh tác cày xới đất quanh gốc để tiêu diệt ấu trùng và nhộng; vệ sinh vườn, làm sạch cỏ dại để giảm nơi trú ẩn. Biện pháp hóa học khi mật độ cao, phun thuốc trừ sâu vào buổi chiều mát thời điểm thành trùng hoạt động mạnh.

#### **4.2.8 Bộ nhảy sọc cong**

Bộ nhảy sọc cong hình 4.11 còn gọi là bộ nhảy sọc cong vỏ lục có tên khoa học *Phyllotreta striolata* (Fabricius). Thành trùng có cơ thể hình bầu dục, kích thước nhỏ, dài khoảng 2-2,5mm, cánh cứng màu đen bóng, giữa mỗi cánh có một vạch màu vàng nhạt cong hình củ lạc rất đặc trưng. Chân sau to khỏe giúp chúng nhảy xa và bay nhanh khi bị động. Mỗi con cái có thể đẻ tới khoảng 200 trứng dưới đất gần gốc cây, trứng nhỏ, màu vàng nhạt. Ấu trùng màu trắng ngà đến vàng nhạt, hình ống, sống trong đất. Vòng đời kéo dài khoảng 47-107 ngày. bộ gây hại mạnh trong các tháng mùa khô [4].



Hình 4.11 Bộ nhảy sọc cong



### ***Xây dựng ứng dụng nhận diện bọ cánh cứng***

---

Bọ nhảy chủ yếu gây hại trên các cây rau họ cải, su hào, súp lơ.... Thành trùng gặm lá và thân tạo thành những lỗ thủng tròn nhỏ; làm lá vàng sớm, cây còi cọc gây hại nặng nhất ở giai đoạn cây con. Ấu trùng sống dưới đất, cắn phá rễ và củ, tạo các đường lổm ngoằn ngoèo hoặc lỗ ăn sâu làm củ và rễ dễ thối. Bọ nhảy là đối tượng khó phòng trừ, có thể gây mất trắng nếu không xử lý kịp thời.

Biện pháp phòng chống: Biện pháp sinh học dùng bẫy dính màu vàng và bắt thủ công vào sáng sớm hoặc chiều mát khi bọ hoạt động. Biện pháp canh tác phơi ải tối thiểu 10-15 ngày trước khi trồng để diệt sâu non và nhộng trong đất; dọn sạch tàn dư vụ trước; luân canh rau với cây trồng khác, đặc biệt là cây trồng nước. Biện pháp hóa học phun thuốc bảo vệ thực vật luân phiên hoạt chất để tránh kháng thuốc, tập trung vào giai đoạn cây con.

#### **4.2.9 Bọ ngâu**



Hình 4.12 Bọ ngâu

Bọ ngâu hình 4.12 là tên gọi dân gian của nhóm bọ cánh cứng ăn lá thuộc các giống *Adoretus* và *Anomala*. Thành trùng màu đen bóng với các sọc xuôi chạy dọc theo cánh, dài trung bình khoảng 10-12 mm, rộng 5-6 mm. Thành trùng có kiểu miệng nhai, ban ngày ẩn nấp dưới lớp đất mặt hoặc trong tán cây rậm, đến chiều tối và ban đêm mới bay ra cắn phá. Ấu trùng là dạng sùng trắng sống trong đất, phổ biến ở nơi đất ẩm, nhiều xác thực vật và chất hữu cơ. Bọ ngâu xuất hiện theo mùa, gây hại nặng nhất vào đầu và cuối mùa mưa, đặc biệt ở những vườn mới khai hoang gần tán rừng [3].

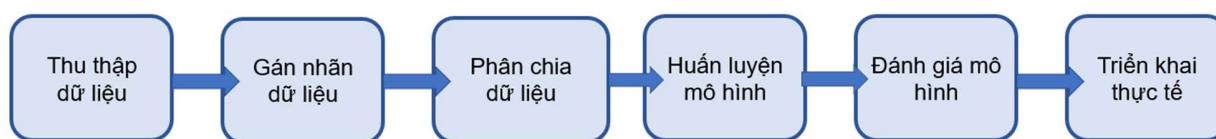
### ***Xây dựng ứng dụng nhận diện bọ cánh cứng***

Bọ ngâu là loài đa thực, gây hại trên nhiều cây ăn trái và cây trồng như sầu riêng, nhãn cùng nhiều loại cây khác. Thành trùng tấn công đốt non, lá non và cả lá đã trưởng thành, gặm khuyết hoặc ăn trụi lá làm thủng lá, suy giảm nghiêm trọng diện tích quang hợp, khiến cây còi cọc; khi mật độ cao chúng có thể cắn phá nặng chỉ trong thời gian ngắn. Vào giai đoạn cây ra hoa, mùi hương dẫn dụ bọ ngâu đến cắn phá bông, gây thối nhũn và rụng bông, ảnh hưởng lớn đến quá trình thụ phấn và đậu trái. Ngoài ra, các vết thương do bọ ngâu gây ra còn tạo điều kiện cho nấm khuẩn xâm nhập. Ở giai đoạn ấu trùng, sùng đất ăn rễ làm cây thiếu dinh dưỡng và suy yếu.

Biện pháp phòng chống: Biện pháp cơ giới dùng bẫy đèn đồng loạt vào chiều tối tại các khu vực xuất hiện nhiều bọ ngâu để thu hút và tiêu diệt thành trùng; kết hợp bắt thủ công. Biện pháp canh tác thường xuyên thăm vườn để phát hiện sớm; đậy kín các đồng ủ phân hữu cơ; cày xới, phơi ải đất để diệt ấu trùng và nhộng. Biện pháp sinh học sử dụng chế phẩm sinh học như *Bacillus thuringiensis* và nấm xanh *Metarhizium anisopliae*; bảo vệ thiên địch tự nhiên. Biện pháp hóa học khi mật độ cao, phun thuốc trừ sâu có hoạt chất phù hợp vào lúc chiều tối.

#### **4.3 Quy trình xây dựng mô hình nhận diện bọ cánh cứng**

Để xây dựng mô hình có khả năng nhận diện các loài bọ cánh cứng, đề tài thực hiện theo một quy trình gồm 6 giai đoạn chính theo hình 4.13. Quy trình bắt đầu từ khâu thu thập, xử lý dữ liệu cho đến huấn luyện và đánh giá mô hình, trong đó kết quả của mỗi giai đoạn là đầu vào cho giai đoạn kế tiếp.



Hình 4.13 Quy trình xây dựng mô hình nhận diện

Thu thập dữ liệu (Data Collection). Hình ảnh các loài bọ cánh cứng được thu thập từ nhiều nguồn khác nhau, gồm iNaturalist, Roboflow Universe, Google Image và ảnh tự chụp bằng camera. Việc kết hợp nhiều nguồn giúp bộ dữ liệu đa dạng về góc chụp, điều kiện ánh sáng và môi trường nền, qua đó nâng cao khả năng khái quát hóa của mô hình.

## ***Xây dựng ứng dụng nhận diện bộ cánh cứng***

---

Gán nhãn dữ liệu (Data Labeling). Dữ liệu thô được sàng lọc để loại bỏ các ảnh trùng lặp, mờ hoặc không đạt chất lượng. Từng cá thể bộ trong ảnh được đóng khung và gán nhãn bằng bounding box, rồi xuất ra theo định dạng chuẩn của YOLO để phục vụ huấn luyện.

Phân chia dữ liệu (Data Splitting). Bộ dữ liệu sau khi gán nhãn được chia thành ba tập theo tỷ lệ, tương ứng tập huấn luyện (train), tập kiểm định (val) và tập kiểm thử (test).

- Tập huấn luyện: dùng để huấn luyện mô hình với một phần lớn dữ liệu. Mô hình sẽ học các đặc điểm của dữ liệu và điều chỉnh tham số dựa trên dữ liệu này.

- Tập xác thực: dùng để điều chỉnh các siêu tham số của mô hình và đánh giá hiệu suất trong quá trình huấn luyện. Tập xác thực không sử dụng để điều chỉnh mô hình mà chỉ để đánh giá hiệu suất.

- Tập kiểm tra: được sử dụng để đánh giá về hiệu suất của mô hình sau khi đã huấn luyện và điều chỉnh siêu tham số. Đây phải là dữ liệu độc lập và không được sử dụng trong quá trình huấn luyện và xác thực.

Huấn luyện mô hình (Model Training). Mô hình được huấn luyện trên nền tảng Google Colab với sự hỗ trợ của GPU và thư viện Ultralytics. Quá trình huấn luyện áp dụng kỹ thuật học chuyển giao (transfer learning) từ trọng số đã được huấn luyện trước trên tập dữ liệu COCO, giúp rút ngắn thời gian hội tụ và tận dụng các đặc trưng thị giác phổ quát.

Đánh giá mô hình (Evaluation). Mô hình sau huấn luyện được đánh giá thông qua các chỉ số mAP, Precision và Recall nhằm xác định mức độ chính xác, từ đó lựa chọn mô hình phù hợp nhất phục vụ triển khai.

Triển khai thực tế (Deployment). Mô hình với trọng số tốt nhất được tích hợp vào hệ thống ứng dụng: nạp lên máy chủ Flask để xử lý nhận diện, còn ứng dụng Flutter đóng vai trò giao diện để người dùng gửi ảnh/video và nhận kết quả.

## **4.4 Thu thập dữ liệu**

### **4.4.1 Nguồn dữ liệu**

Để xây dựng nguồn dữ liệu phong phú và đa dạng dữ liệu hình ảnh được thu thập từ nhiều nguồn dữ liệu khác nhau như: iNaturalist, Roboflow Universe, Google Image và hình



### ***Xây dựng ứng dụng nhận diện bộ cánh cứng***

ảnh tự thu thập bằng camera. Sử dụng kết hợp nhiều nguồn giúp tăng sự đa dạng về số lượng, góc độ và môi trường nền của ảnh trong bộ dữ liệu.

#### **a) iNaturalist**

iNaturalist là nền tảng khoa học cộng đồng, nơi người dùng trên khắp thế giới đăng tải quan sát động vật ngoài tự nhiên kèm định danh loài để xác nhận bởi chuyên gia. Ảnh từ iNaturalist được chụp trong nhiều điều kiện tự nhiên với các góc độ và môi trường nền khác nhau, giúp mô hình học với hình ảnh đa dạng và thực tế nhất.

#### **b) Roboflow Universe**

Roboflow Universe là kho dữ liệu thị giác máy tính mã nguồn mở lớn nhất hiện nay, nơi cộng đồng nghiên cứu toàn cầu chia sẻ hàng trăm nghìn tập dữ liệu đã được gán nhãn sẵn. Từ nguồn này có thể tải ảnh đã được thu thập hoặc tải thẳng các bộ dữ liệu dataset bộ cánh cứng đã có annotation dạng bounding box theo định dạng YOLO.

#### **c) Google Image**

Google Images được sử dụng để bổ sung ảnh cho các lớp loài còn thiếu mẫu sau khi thu thập từ các nguồn trên. Quá trình tìm kiếm thực hiện thủ công bằng tên loài. Mỗi ảnh được kiểm tra kỹ về chất lượng và tính chính xác về loài trước khi đưa vào bộ dữ liệu.

#### **d) Ảnh tự thu thập bằng camera**

Ngoài các nguồn trực tuyến, một phần ảnh được thu thập bằng cách trực tiếp chụp bộ cánh cứng ngoài thực địa tại địa phương. Ảnh tự chụp có ưu điểm là phản ánh đúng điều kiện ánh sáng, nền và góc độ thực tế mà người dùng sẽ gặp khi sử dụng ứng dụng, giúp mô hình thích nghi tốt hơn với dữ liệu thực.

### **4.4.2 Số lượng và phân phối dữ liệu theo lớp**

Sau khi hoàn tất quá trình thu thập, bộ dữ liệu gồm 2648 ảnh phân bố trên 9 lớp loài bộ cánh cứng.

Bảng 4-1 Phân phối dữ liệu theo lớp

| <b>Tên lớp</b> | <b>Tên loài</b>    | <b>Tên khoa học</b>       | <b>Tổng</b> |
|----------------|--------------------|---------------------------|-------------|
| BoDua          | Bọ dừa             | Brontispa longissima      | 320         |
| VoiVoi         | Bọ vòi voi hại dừa | Diocalandra frumenti      | 299         |
| DuongDua       | Đuông dừa          | Rhynchophorus ferrugineus | 286         |

### ***Xây dựng ứng dụng nhận diện bộ cánh cứng***

| Tên lớp          | Tên loài                | Tên khoa học                             | Tổng |
|------------------|-------------------------|------------------------------------------|------|
| KienVuong        | Kiến vương / Kiến dương | Oryctes rhinoceros                       | 315  |
| BoHa             | Bộ hà khoai lang        | Cylas formicarius                        | 274  |
| BoRay            | Bộ rầy / Bù rầy         | Holotrichia sauteri                      | 238  |
| CauCau           | Câu cầu xanh            | Hypomeces squamosus                      | 309  |
| BoNhay           | Bộ nhảy sọc cong        | Phyllotreta striolata                    | 298  |
| BoNgau           | Bộ ngâu                 | Adoretus <i>sp.</i> / Anomala <i>sp.</i> | 309  |
| <b>Tổng cộng</b> |                         |                                          | 2648 |

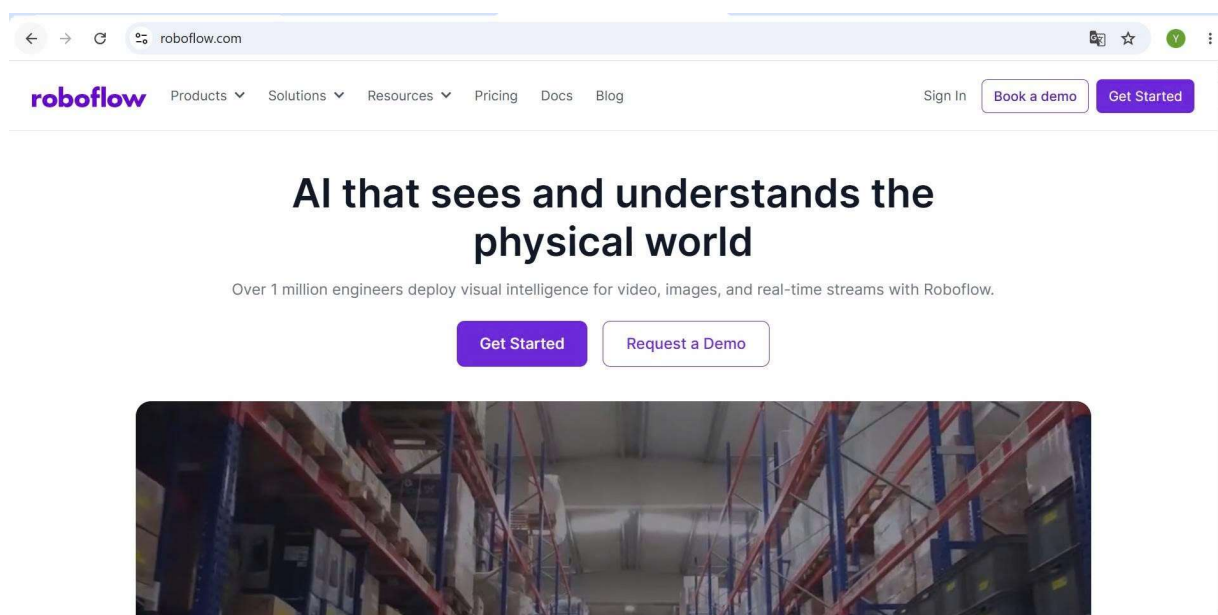
Nhìn vào bảng 4-1 có thể thấy số lượng ảnh giữa các lớp tương đối đồng đều, dao động trong khoảng 238 đến 320 mỗi lớp. Sự cân bằng này giúp hạn chế hiện tượng mất cân bằng lớp tránh cho mô hình bị thiên lệch về phía các lớp có nhiều dữ liệu hơn và giảm khả năng nhận diện các lớp ít dữ liệu hơn.

## **4.5 Xử lý dữ liệu**

### **4.5.1 Quy trình đóng khung và gán nhãn**

a) Khởi tạo dự án trên Roboflow

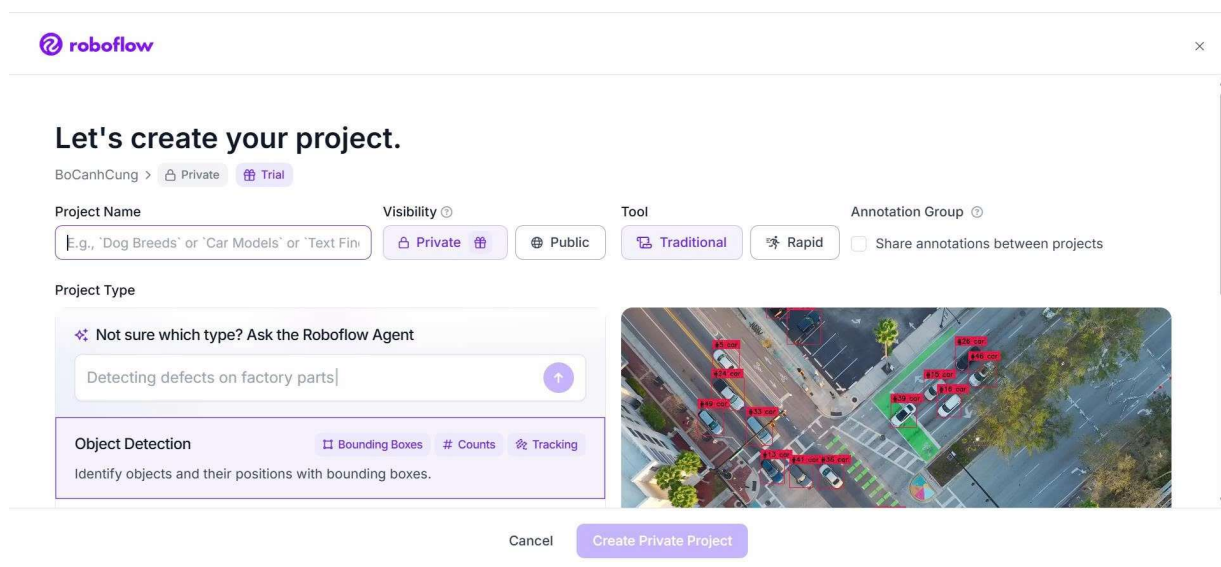
Trước tiên, truy cập trang roboflow.com và đăng nhập tài khoản để bắt đầu sử dụng nền tảng.



Hình 4.14 Giao diện trang chủ Roboflow

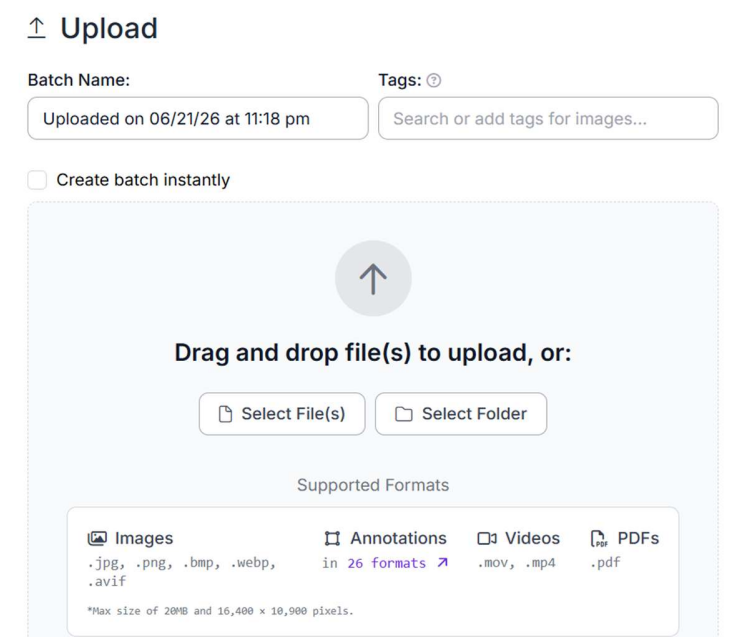
## Xây dựng ứng dụng nhận diện bộ cánh cứng

Sau khi đăng nhập, tiến hành tạo một dự án mới trong workspace. Tại bước này cần thiết lập các thông tin cho dự án đặt tên dự án, chọn chế độ riêng tư, công cụ gán nhãn và đặc biệt là loại dự án. Do bài toán của đề tài là xác định vị trí và phân loại các cá thể bộ cánh cứng trong ảnh nên loại dự án được chọn là Object Detection với phương pháp gán nhãn bằng khung bao Bounding Boxes.



Hình 4.15 Khởi tạo dự án Object Detection trên Roboflow

### b) Gán nhãn dữ liệu

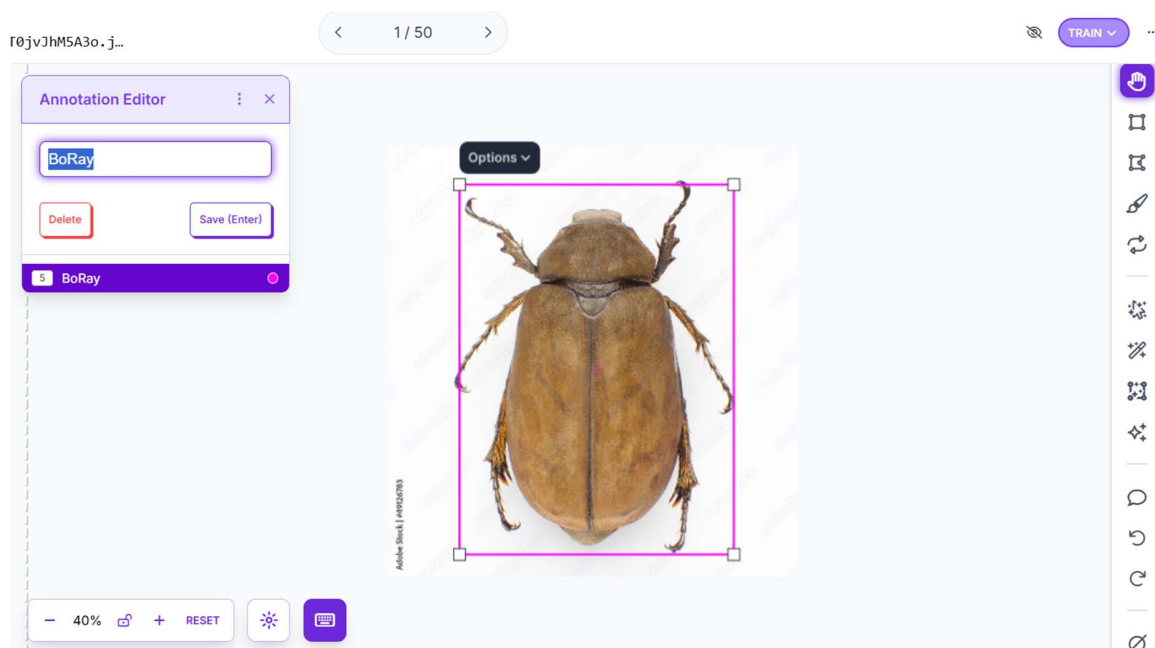


Hình 4.16 Tải dữ liệu lên Roboflow

## ***Xây dựng ứng dụng nhận diện bọ cánh cứng***

Sau khi tạo dự án, toàn bộ ảnh đã thu thập được tải lên Roboflow như hình 4.16 và tiến hành gán nhãn. Với mỗi ảnh sử dụng công cụ vẽ khung để khoanh vùng bao quanh từng cá thể bọ cánh cứng xuất hiện trong ảnh, sau đó gán nhãn lớp tương ứng với loài của cá thể đó.

Quá trình gán nhãn được thực hiện thông qua Annotation Editor người gán nhãn vẽ khung, chọn hoặc nhập tên lớp rồi lưu lại. Ngoài việc gán nhãn thủ công, Roboflow còn cung cấp tính năng Find Objects with AI giúp tự động phát hiện và đề xuất khung bao cho đối tượng, qua đó tăng tốc đáng kể quá trình gán nhãn so với vẽ thủ công hoàn toàn. Mỗi ảnh sau khi gán nhãn xong được kiểm tra lại để đảm bảo khung bao ôm sát đối tượng và đúng lớp.



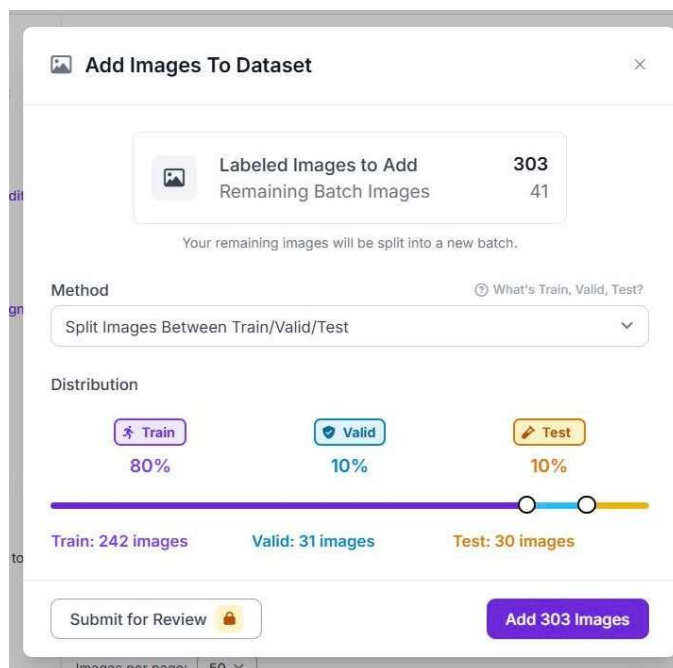
Hình 4.17 Gán nhãn đối tượng bằng bounding box trên Roboflow

### **4.5.2 Chia dữ liệu**

Trong học máy, để đánh giá hiệu suất của một mô hình thì cần quan sát kết quả mô hình trên các dữ liệu mới. Chia tập dữ liệu gốc thành các phần riêng biệt cho huấn luyện (training), xác thực (validation) và kiểm tra (test).

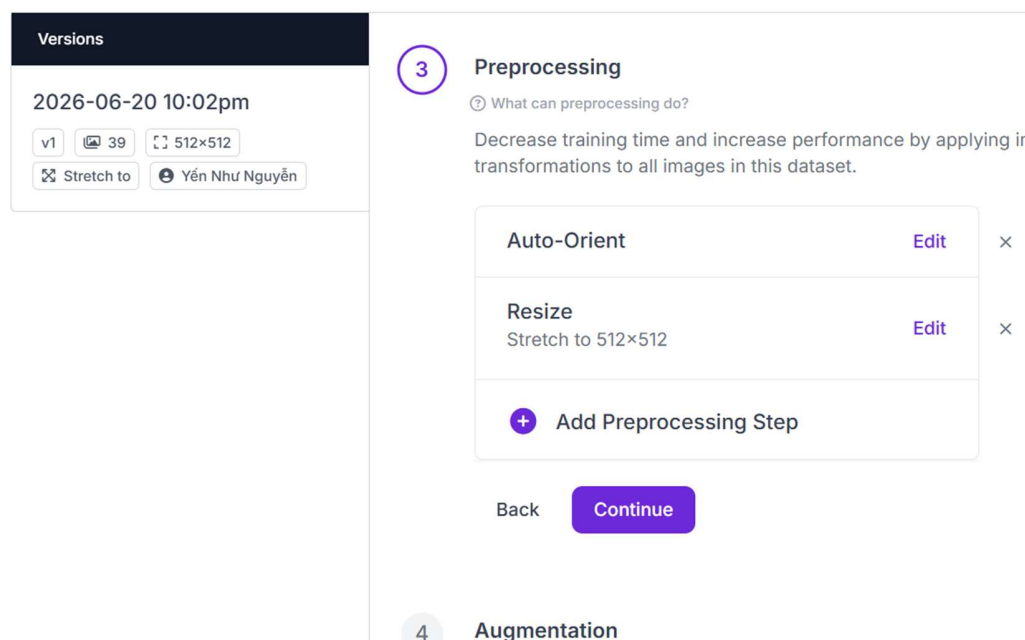
## Xây dựng ứng dụng nhận diện bộ cánh cứng

Trong đề tài này sử dụng tỷ lệ 80-10-10. Các ảnh được chọn ngẫu nhiên vào các tập trên, đảm bảo tính tổng quát của mô hình.



Hình 4.18 Chia dữ liệu

### 4.5.3 Tăng cường dữ liệu



Hình 4.19 Chọn phép tăng cường dữ liệu

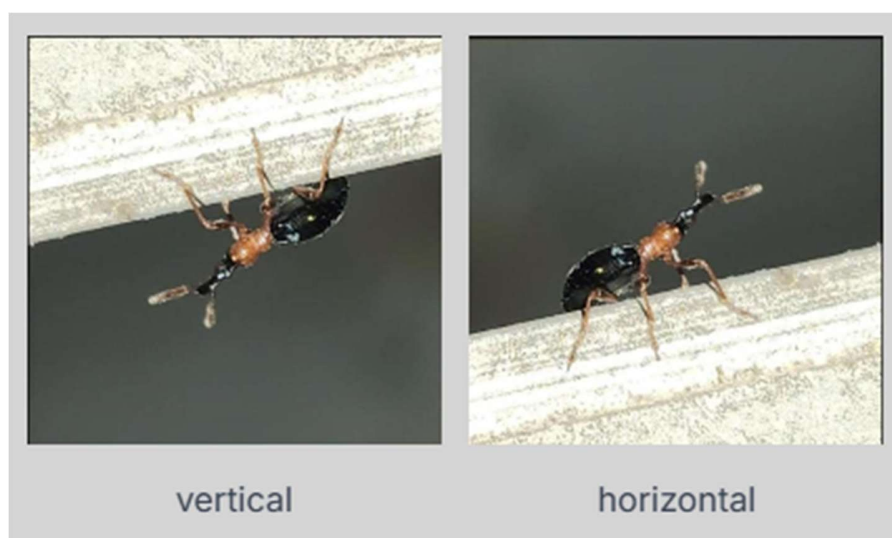
### ***Xây dựng ứng dụng nhận diện bọ cánh cứng***

---

Trong học sâu, tăng cường hình ảnh là kỹ thuật quan trọng để tăng cường và biến đổi tập dữ liệu gốc. Mục đích chính của tăng cường hình ảnh là tạo ra nhiều phiên bản khác nhau của các hình ảnh ban đầu mà không thay đổi nội dung quan trọng của chúng. Kỹ thuật này giúp cải thiện khả năng tổng quát hóa của mô hình và quá trình học bằng cách thay đổi góc nhìn, điều kiện ánh sáng, hướng và tỷ lệ của các hình ảnh

Các phép tăng cường dữ liệu được sử dụng:

Lật ảnh (Flip) như trong hình 4.20 : Lật ảnh theo chiều dọc và chiều ngang.



Hình 4.20 Ảnh dùng Flip

Cắt ảnh (Crop): Cắt một vùng rồi phóng to vùng đó về kích thước tiêu chuẩn của mô hình.



Hình 4.21 Ảnh dùng Crop

Xoay ảnh (Rotation) như trong hình 4.22: Bức ảnh sẽ được xoay một góc tùy chỉnh.



Hình 4.22 Ảnh dùng Rotation

## **4.6 Huấn luyện mô hình**

### **4.6.1 Môi trường huấn luyện**

Quá trình huấn luyện mô hình được thực hiện trên nền tảng Google Colaboratory (Colab). Đây là dịch vụ điện toán đám mây do Google cung cấp, cho phép chạy mã nguồn Python trực tiếp trên trình duyệt và sử dụng miễn phí bộ xử lý đồ họa. Việc lựa chọn Google Colab giúp tận dụng được tài nguyên tính toán mạnh mà không cần đầu tư máy tính cấu hình cao tại chỗ, vốn là một hạn chế thường gặp đối với đề tài của sinh viên.

Về phần cứng, mô hình được huấn luyện trên bộ xử lý đồ họa NVIDIA Tesla T4 dựa trên kiến trúc Turing, với dung lượng bộ nhớ khả dụng khoảng 15 GB trong môi trường Colab, cùng bộ nhớ RAM hệ thống khoảng 12,7 GB. Loại GPU này hỗ trợ tính toán song song và tích hợp các nhân Tensor chuyên dụng, giúp tăng tốc đáng kể quá trình huấn luyện mô hình học sâu. Với cấu hình này, GPU đáp ứng tốt yêu cầu huấn luyện biến thể YOLO26n vốn có kích thước nhỏ gọn, phù hợp với mục tiêu triển khai trên thiết bị di động của đề tài.

Về môi trường phần mềm, mô hình được xây dựng và huấn luyện bằng ngôn ngữ lập trình Python phiên bản 3.12.13, sử dụng thư viện Ultralytics phiên bản 8.4.72 làm framework chính cho việc huấn luyện và đánh giá. Thư viện này hoạt động dựa trên nền



## ***Xây dựng ứng dụng nhận diện bọ cánh cứng***

tăng học sâu PyTorch phiên bản 2.11.0, kết hợp với bộ công cụ CUDA phiên bản 12.8 để khai thác khả năng tính toán của GPU.

Bảng 4-2 Cấu hình môi trường huấn luyện

| Thành phần                 | Cấu hình                    |
|----------------------------|-----------------------------|
| Nền tảng huấn luyện        | Google Colaboratory (Colab) |
| Ngôn ngữ lập trình         | Python 3.12.13              |
| Bộ xử lý đồ họa (GPU)      | NVIDIA Tesla T4             |
| Nền tảng học sâu           | PyTorch 2.11.0              |
| Bộ công cụ tính toán GPU   | CUDA 12.8                   |
| Framework huấn luyện       | Ultralytics 8.4.72          |
| Lưu trữ dữ liệu và mô hình | Google Drive                |

### **4.6.2 Huấn luyện mô hình**

Đề tài lựa chọn mô hình YOLO26n để huấn luyện. Quá trình huấn luyện được thực hiện trên Google Colab bằng thư viện Ultralytics. Phần này trình bày các đoạn mã chính cùng giải thích chức năng của từng bước.

#### **Bước 1: Chuẩn bị bộ dữ liệu và cài đặt thư viện**

```
!unzip BeetleDetectionv5.zip -d my_dataset  
  
!pip install ultralytics
```

Lệnh unzip giải nén tệp nén BeetleDetectionv5.zip đây là bộ dữ liệu đã được gán nhãn và chia tập trên Roboflow vào thư mục my\_dataset. Sau khi giải nén, thư mục này chứa các tập ảnh huấn luyện, kiểm định cùng tệp cấu hình data.yaml. Lệnh pip install ultralytics cài đặt thư viện Ultralytics thư viện cung cấp sẵn mô hình YOLO và các công cụ huấn luyện, đánh giá mô hình.

#### **Bước 2: Khởi tạo và huấn luyện mô hình**

```
from ultralytics import YOLO  
model = YOLO("yolo26n.pt")  
model.train(  
    data="/content/my_dataset/data.yaml",  
    epochs=50,
```



## ***Xây dựng ứng dụng nhận diện bộ cánh cứng***

```
imgsz=640,  
batch=-1,  
cache=True,  
)
```

Sử dụng kỹ thuật học chuyển giao mô hình kế thừa các trọng số đã được huấn luyện sẵn trên tập dữ liệu lớn COCO thay vì khởi tạo ngẫu nhiên, nhờ đó hội tụ nhanh hơn và đạt độ chính xác cao hơn trên bộ dữ liệu quy mô vừa phải của đề tài.

### **4.7 Kết quả huấn luyện**

#### **4.7.1 Kết quả tổng thể**

Sau 60 epoch huấn luyện, mô hình đạt kết quả tốt trên tập kiểm định. Các chỉ số đánh giá của bộ trọng số tốt nhất như sau:

Bảng 4-3 Kết quả huấn luyện tổng thể

| <b>Chỉ số</b> | <b>Giá trị</b> |
|---------------|----------------|
| Precision     | 0,947          |
| Recall        | 0,923          |
| mAP@0.5       | 0,947          |
| mAP@0.5:0.95  | 0,816          |

Kết quả huấn luyện trong bảng 4-3 cho thấy độ chính xác (Precision) đạt 0,947, nghĩa là trong những lần mô hình khoanh vùng và khẳng định một đối tượng thuộc một loài nào đó thì có 94,7% là đúng. Độ bao phủ (Recall) đạt 0,923, cho thấy mô hình phát hiện được khoảng 92,3% số đối tượng thực tế xuất hiện trong ảnh. Về các chỉ số tổng hợp, mAP@0.5 đạt 0,947, mAP@0.5:0.95 đạt 0,816.

#### **4.7.2 Kết quả theo từng loài**

Bảng 4-4 Kết quả huấn luyện theo từng loài

| <b>Mã lớp</b> | <b>Tên loài</b> | <b>AP@0.5</b> |
|---------------|-----------------|---------------|
| BoDua         | Bọ dừa          | 0,894         |
| BoHa          | Bọ hà           | 0,935         |
| BoNgau        | Bọ ngâu         | 0,990         |

### ***Xây dựng ứng dụng nhận diện bọ cánh cứng***

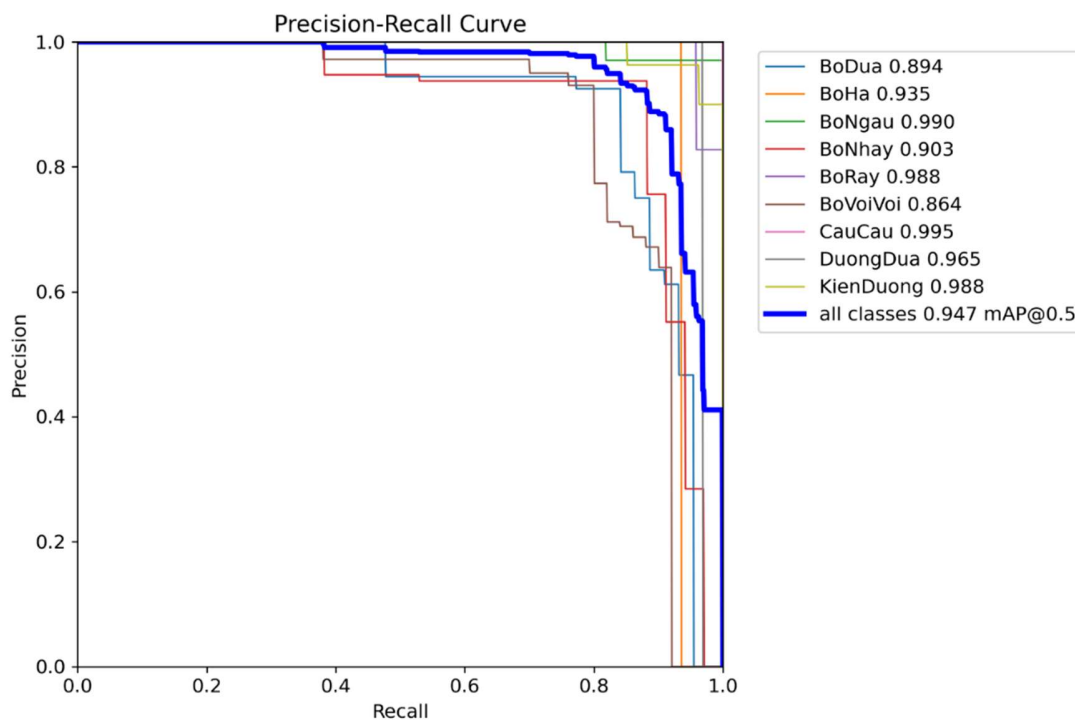
|                             |                  |              |
|-----------------------------|------------------|--------------|
| BoNhay                      | Bọ nhảy sọc cong | 0,903        |
| BoRay                       | Bọ rầy           | 0,988        |
| BoVoiVoi                    | Bọ vôi vôi       | 0,864        |
| CauCau                      | Câu cầu xanh     | 0,995        |
| DuongDua                    | Đuông dừa        | 0,965        |
| KienDuong                   | Kiến vương       | 0,988        |
| <b>Trung bình (mAP@0.5)</b> |                  | <b>0,947</b> |

Dựa vào bảng 4-4 có thể thấy hầu hết các loài đều đạt trên 0.9. Loài nhận diện tốt nhất là câu cầu xanh 0,995, còn hai loài có kết quả thấp hơn là bọ vôi vôi 0,864 và bọ dừa 0,894. Nhìn chung, các giá trị này cho thấy mô hình đạt hiệu năng tốt, vừa có độ chính xác cao vừa ít bỏ sót đối tượng, đáp ứng được mục tiêu đề ra của đề tài.

#### **4.7.3 Các biểu đồ kết quả**

##### **a) Đường cong Precision-Recall**

*Bảng 4-5*

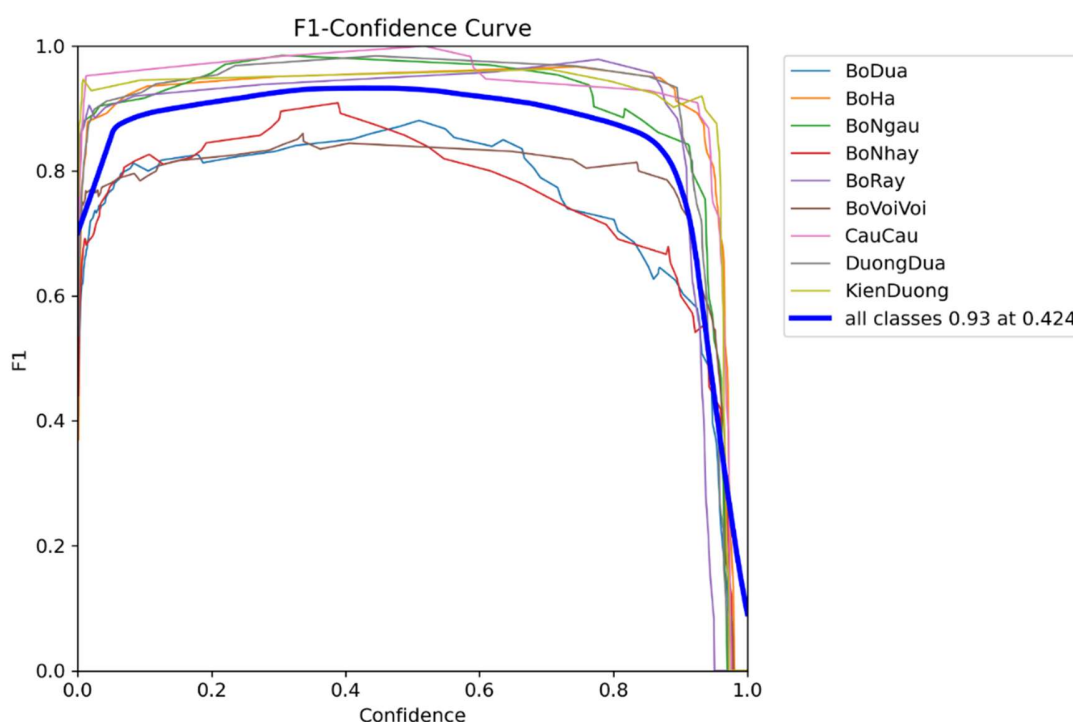


**Hình 4.23 Biểu đồ đường cong Precision–Recall**

Đây là biểu đồ quan trọng nhất để đánh giá tổng thể mô hình. Nó cho thấy mô hình giữ được độ chính xác cao tới mức nào khi cố gắng bao phủ ngày càng nhiều đối tượng. Một mô hình tốt sẽ có đường cong bám sát góc trên bên phải là mô hình vừa chính xác vừa bao phủ rộng, và diện tích nằm dưới đường cong chính là chỉ số mAP. Theo hình 4.23 đường cong của hầu hết các loài đều nằm cao và sát góc phải, ứng với  $mAP@0.5$  chung đạt 0,947. Trong đó câu cáu xanh (0,995), bộ ngẫu (0,990) và bộ rầy (0,988) đạt gần như tuyệt đối, còn bộ vôi voi (0,864) là loài có đường cong thấp nhất.

### **b) Đường cong F1-Confidence**

F1 là trung bình điều hòa của Precision và Recall, phản ánh mức độ cân bằng giữa độ chính xác và độ bao phủ của mô hình. Mỗi dự đoán do mô hình đưa ra được gán một giá trị độ tin cậy trong khoảng từ 0 đến 1. Theo kết quả thể hiện trên biểu đồ hình 4.24, chỉ số F1 của toàn bộ các lớp đạt giá trị cực đại bằng 0,93 tại ngưỡng tin cậy 0,424. Đây là ngưỡng cho kết quả cân bằng tốt nhất giữa Precision và Recall, có thể được sử dụng làm cơ sở để thiết lập ngưỡng tin cậy cho mô hình khi triển khai trong ứng dụng thực tế.

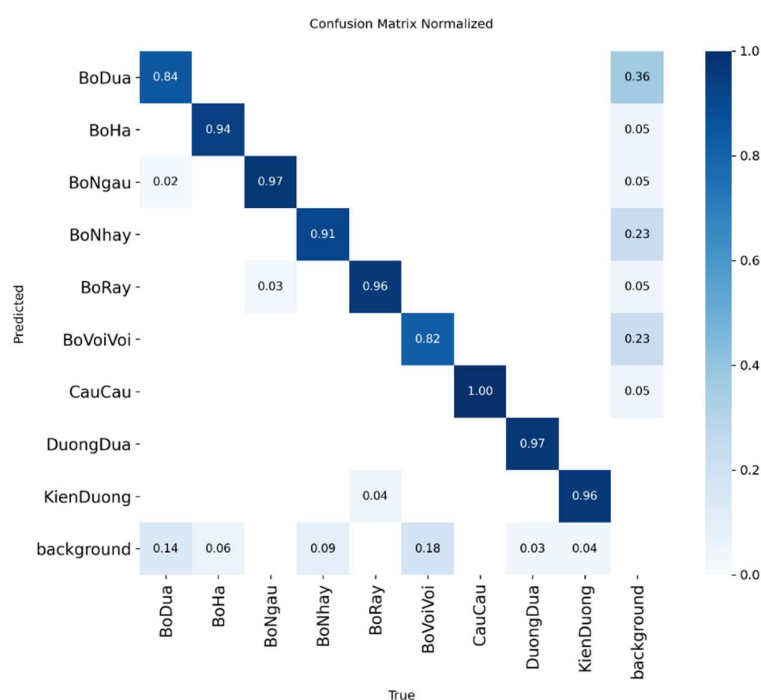


Hình 4.24 Biểu đồ đường cong F1 - Confidence

**c) Ma trận nhầm lẫn**

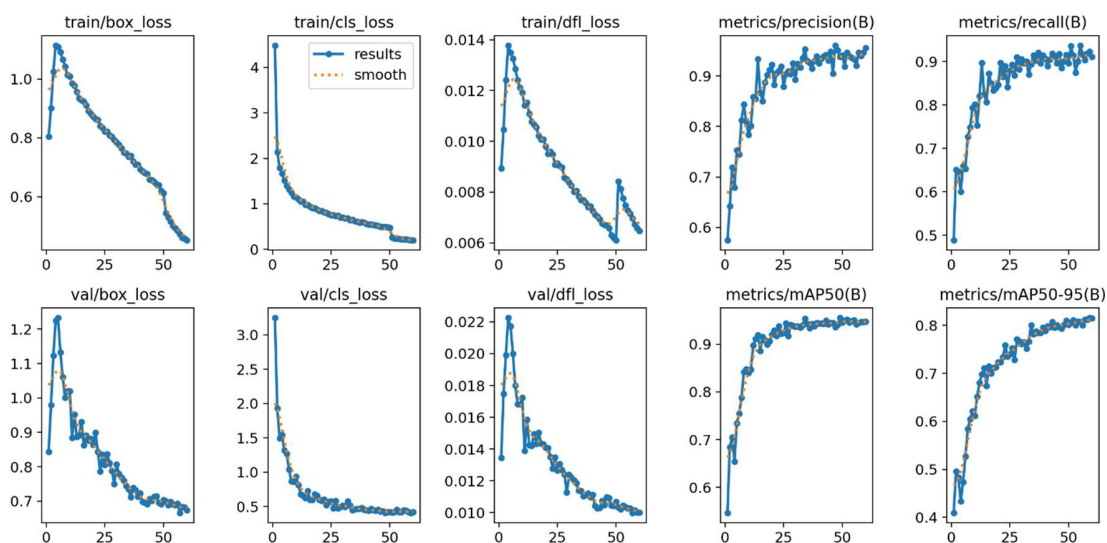
Ma trận nhầm lẫn cho biết mô hình hay nhầm loài nào với loài nào. Cách đọc cột là loài thật, hàng là loài mà mô hình đoán; ô nằm trên đường chéo là những trường hợp đoán đúng, nên đường chéo càng đậm thì mô hình càng tốt. Hàng background là những con bọ có thật nhưng mô hình không phát hiện được, còn cột background là những vùng nền trống bị mô hình tưởng nhầm là con bọ.

Dựa vào ma trận nhầm lẫn hình 4.25 có thể thấy đa số loài được phân loại đúng trên 90%, riêng câu câu xanh đúng hoàn toàn (100%). Hai loài thấp hơn là bọ dừa (84%) và bọ vòi voi (82%). Điều đáng chú ý là phần sai của hai loài này gần như không phải do nhầm sang loài khác, mà rơi vào hàng background tức là mô hình bỏ sót, không nhận ra con bọ.



Hình 4.25 Ma trận nhầm lẫn

**d) Biểu đồ kết quả theo từng epoch**



**Hình 4.26 Biểu đồ kết quả theo từng epoch**

Hình 4.26 gồm mười đồ thị nhỏ ghi lại quá trình học của mô hình qua 60 epoch; trên mỗi đồ thị, đường liền màu xanh (results) là giá trị thực tế và đường nét đứt màu cam (smooth) thể hiện xu hướng chung. Hàng trên gồm ba đường mất mát trên tập huấn luyện cùng hai chỉ số Precision và Recall; hàng dưới gồm ba đường mất mát tương ứng trên tập kiểm định cùng hai chỉ số mAP@0.5 và mAP@0.5:0.95.

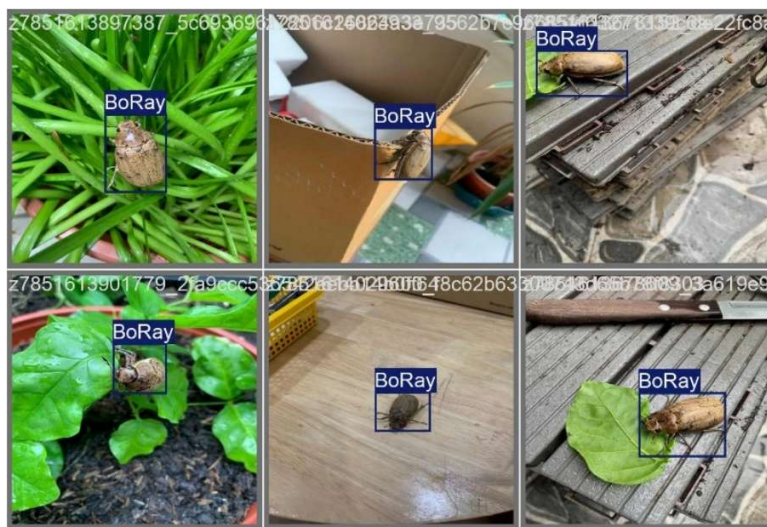
Các đường mất mát phản ánh mức độ sai lệch của mô hình, giá trị càng giảm thì mô hình học càng tốt. Trên tập huấn luyện, cả ba đường mất mát đều giảm rõ rệt qua 60 epoch: box\_loss giảm từ khoảng 0,80 xuống còn 0,45; cls\_loss giảm mạnh từ 4,49 xuống 0,20; và dfl\_loss giảm từ 0,0089 xuống 0,0065. Trên tập kiểm định box\_loss giảm từ 0,84 xuống 0,67; cls\_loss giảm từ 3,25 xuống 0,42; và dfl\_loss giảm từ 0,0134 xuống 0,0100. Việc mất mát trên tập kiểm định giảm và không tăng trở lại ở giai đoạn cuối cho thấy mô hình học được đặc trưng tổng quát chứ không bị overfitting.

Song song với việc mất mát giảm, bốn chỉ số đánh giá đều tăng dần rồi ổn định ở mức cao. Precision tăng từ khoảng 0,57 lên xấp xỉ 0,95; Recall tăng từ 0,49 lên khoảng 0,91; mAP@0.5 tăng từ 0,55 lên 0,947; và mAP@0.5:0.95 tăng từ 0,41 lên khoảng 0,82. Các chỉ số tăng nhanh trong khoảng 15-20 epoch đầu, sau đó tăng chậm dần và đi vào ổn định, cho thấy mô hình đã hội tụ.

## ***Xây dựng ứng dụng nhận diện bọ cánh cứng***

### **e) So sánh nhãn thực tế và kết quả dự đoán**

Dựa vào hình 4.27 hình được gán nhãn ban đầu và hình 24.8 kết quả mô hình huấn luyện được kết quả cho thấy các khung bao và nhãn loài do mô hình đưa ra gần như trùng khớp với nhãn thực tế, với độ tin cậy phần lớn đạt rất cao, kể cả đối với các ảnh được chụp trong điều kiện thực tế có nền phức tạp. Điều này cho thấy mô hình hoạt động ổn định và có khả năng áp dụng tốt vào thực tiễn.



**Hình 4.27 Ảnh được gán nhãn**



**Hình 4.28 Ảnh kết quả dự đoán**

#### **4.7.4 Nhận xét**

Mô hình YOLO26n đạt kết quả khả quan với  $mAP@0.5$  là 0,947 và  $mAP@0.5:0.95$  là 0,816, đủ tốt để triển khai vào ứng dụng nhận diện bọ cánh cứng trên thiết bị di động. Các biểu đồ đều cho thấy mô hình học ổn định, không quá khớp, và hoạt động tốt trên ảnh thực tế. Hạn chế lớn nhất nằm ở hai loài bọ vòi voi và bọ dừa, vốn dễ bị bỏ sót do hình thái lẫn với nền; đây là hướng có thể cải thiện trong tương lai bằng cách thu thập thêm ảnh đa dạng cho hai loài này, đặc biệt là ảnh trên nhiều loại nền khác nhau.

## **Chương 5: XÂY DỰNG HỆ THỐNG NHẬN DIỆN BỌ CÁNH CỨNG**

Nội dung chương bao gồm yêu cầu chức năng hệ thống, sau đó lần lượt trình bày kiến trúc tổng thể của hệ thống, quá trình xây dựng phía backend cùng cơ sở dữ liệu, và thiết kế giao diện ứng dụng phía client. Mục tiêu của chương là hiện thực hóa một hệ thống hoàn chỉnh đưa mô hình đã huấn luyện vào sử dụng thực tế.

### **5.1 Yêu cầu chức năng**

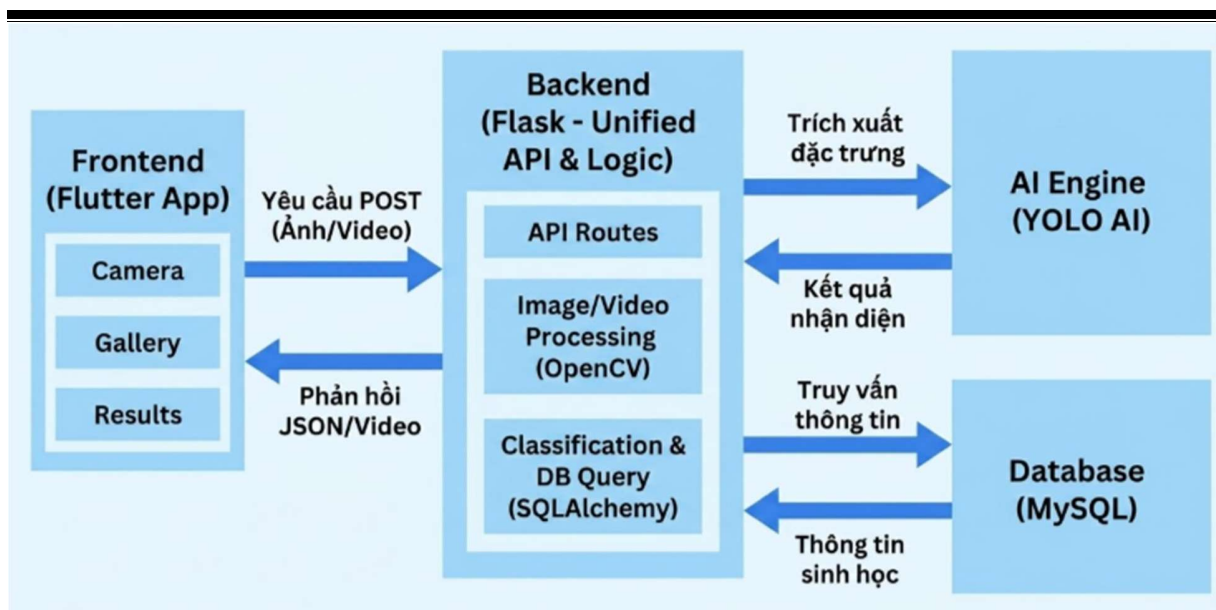
Yêu cầu của hệ thống là xây dựng một ứng dụng cho phép người dùng cung cấp đầu vào là ảnh hoặc video, gửi lên máy chủ để mô hình xử lý và trả về kết quả nhận diện. Với mỗi ảnh, hệ thống cần phát hiện toàn bộ các cá thể bọ cánh cứng xuất hiện trong ảnh; ứng với mỗi cá thể, kết quả trả về gồm nhãn loài được dự đoán, độ tin cậy của dự đoán và tọa độ bounding box xác định vị trí của cá thể đó trong ảnh. Trên cơ sở nhãn loài nhận được, hệ thống truy vấn cơ sở dữ liệu thông tin loài để bổ sung các thông tin sinh thái tương ứng gồm tên tiếng Việt, tên khoa học, họ, kích thước, màu sắc, môi trường sống, mức độ gây hại và biện pháp phòng chống.

Đối với đầu vào là video, do video là một chuỗi gồm nhiều khung hình liên tiếp nên hệ thống xử lý lần lượt từng khung hình, áp dụng mô hình nhận diện trên mỗi khung, vẽ khung bao lên những khung đã phát hiện được đối tượng, sau đó ghép lại thành video kết quả và tổng hợp danh sách các loài đã xuất hiện kèm số lần phát hiện. Như vậy, so với nhận diện ảnh, nhận diện video đòi hỏi thêm bước tách và xử lý khung hình cùng việc tổng hợp kết quả trên toàn bộ video.

### **5.2 Kiến trúc hệ thống**

Hệ thống nhận diện bọ cánh cứng được xây dựng theo mô hình client-server gồm ba thành phần ứng dụng di động (client), máy chủ tích hợp trí tuệ nhân tạo (server) và cơ sở dữ liệu (database). Ba thành phần này hoạt động phối hợp với nhau theo cơ chế yêu cầu và phản hồi client gửi dữ liệu đầu vào lên server, server xử lý nhận diện và truy vấn cơ sở dữ liệu, sau đó trả kết quả hoàn chỉnh về cho client để hiển thị.





Hình 5.1 Kiến trúc hệ thống

### 5.2.1 Các thành phần chính

Dựa vào hình 5.1 cho thấy kiến trúc hệ thống gồm 3 phần chính như sau:

Ứng dụng di động (client) được phát triển bằng Flutter, là nơi người dùng trực tiếp tương tác. Client đảm nhận việc thu thập dữ liệu đầu vào, gồm chụp ảnh hoặc quay video bọ cánh cứng, và hiển thị kết quả cuối cùng cho người dùng.

Máy chủ (server) được xây dựng bằng Flask kết hợp mô hình trí tuệ nhân tạo YOLO26n, đóng vai trò cầu nối trung tâm của hệ thống. Server vừa xử lý logic của các API, vừa đảm nhận tác vụ nặng là chạy mô hình nhận diện hình ảnh. Đây là nơi tập trung toàn bộ phần xử lý nghiệp vụ của hệ thống.

Cơ sở dữ liệu (database) sử dụng hệ quản trị MySQL, lưu trữ bộ thông tin sinh thái của toàn bộ các loài bọ cánh cứng, gồm tên tiếng Việt, tên khoa học, họ, kích thước, tập tính, mức độ gây hại và biện pháp phòng chống. Dữ liệu trong cơ sở dữ liệu mang tính tra cứu, đóng vai trò như một bộ từ điển để bổ sung thông tin cho kết quả nhận diện.

### 5.2.2 Luồng hoạt động của hệ thống

Một chu trình hoạt động hoàn chỉnh của hệ thống diễn ra theo hai pha rõ rệt gồm pha yêu cầu và pha trả về.

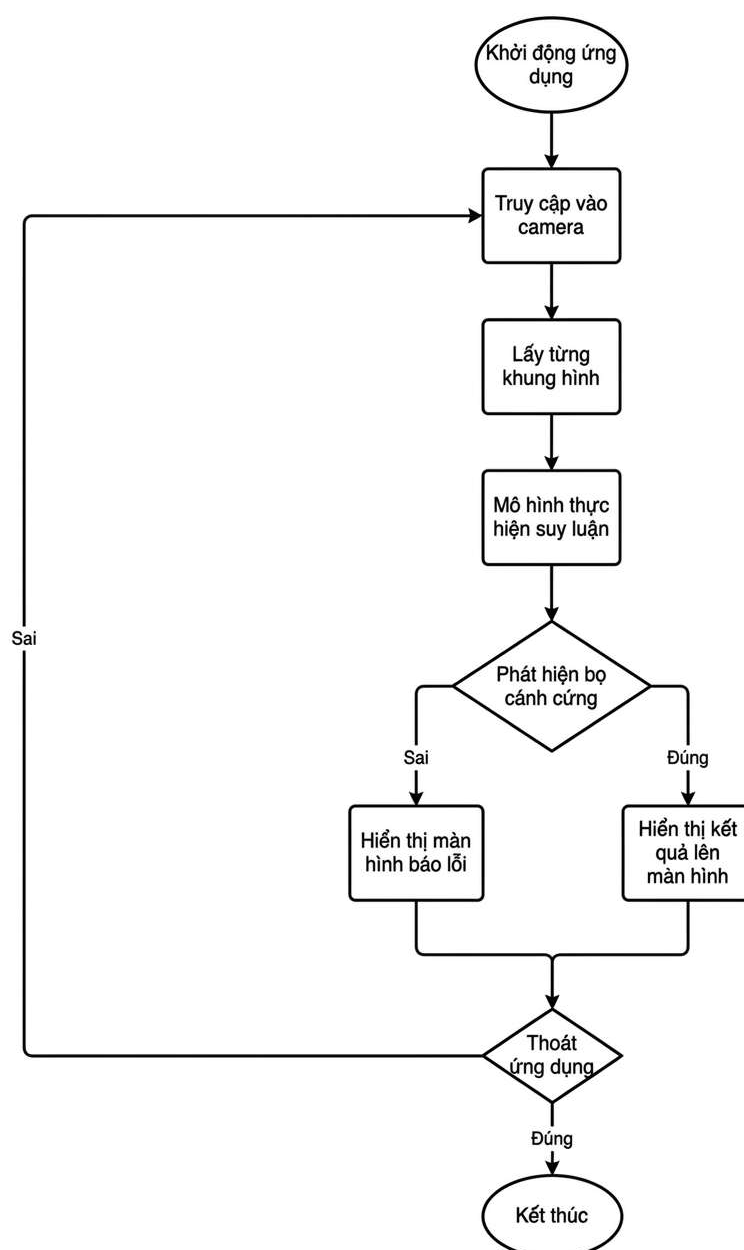
Ở pha yêu cầu, dữ liệu được luân chuyển theo chiều từ client đến server rồi xuống database. Khi người dùng tải lên một ảnh hoặc video, ứng dụng Flutter đóng gói dữ liệu dưới dạng multipart/form-data và gửi một yêu cầu HTTP POST đến server. Tại server, Flask tiếp nhận tệp và chuyển vào mô-đun nhận diện. Mô hình YOLO26n quét ảnh, phát hiện các cá thể bộ và xuất ra hai thông tin cốt lõi tọa độ bounding box xác định vị trí của cá thể và nhãn loài tương ứng. Có được nhãn loài, server chưa trả kết quả ngay mà sử dụng ORM để tạo một câu truy vấn xuống cơ sở dữ liệu nhằm tìm bản ghi có nhãn loài trùng khớp.

Ở pha trả về, dữ liệu đi theo chiều ngược lại, từ database qua server rồi về client. Cơ sở dữ liệu tìm thấy và trả về cho server toàn bộ bản ghi sinh học chi tiết của loài, gồm tên tiếng Việt, họ, kích thước, tập tính, mức độ gây hại và biện pháp phòng chống. Server tiến hành tổng hợp dữ liệu ghép tọa độ bounding box nhận được từ mô hình AI với thông tin sinh học lấy từ cơ sở dữ liệu, đóng gói tất cả thành một cấu trúc JSON thống nhất. Riêng đối với video, server thực hiện thêm bước vẽ khung bao trực tiếp lên các khung hình, xuất ra tệp mp4 theo chuẩn nén H.264, rồi tạo một đường dẫn URL cho tệp video kết quả. Cuối cùng, server gửi gói tin JSON, kèm URL video nếu có, trở lại cho ứng dụng thông qua phản hồi HTTP. Ứng dụng Flutter bóc tách gói JSON, dùng tọa độ khung bao để vẽ khung lên ảnh hiển thị trên màn hình, đồng thời trình bày các thông tin sinh học của loài ở bên dưới để người dùng tham khảo.

Mô hình trí tuệ nhân tạo chỉ đảm nhận nhiệm vụ ra quyết định phân loại, còn cơ sở dữ liệu đóng vai trò là nguồn tri thức để bổ sung thông tin hữu ích cho người dùng. Chính sự liên kết giữa ba thành phần client, server và database tạo nên giá trị sử dụng thực tế của hệ thống, người dùng không chỉ biết được loài bộ trong ảnh mà còn nhận được đầy đủ thông tin sinh thái và cách phòng trị tương ứng.

### **5.3 Hoạt động chức năng hệ thống**

Quá trình nhận diện bắt đầu khi người dùng cung cấp dữ liệu đầu vào cho ứng dụng. Người dùng chọn một ảnh hoặc một video từ thiết bị, hoặc chụp và quay trực tiếp, rồi tải lên máy chủ. Tùy theo dạng dữ liệu là ảnh hay video, máy chủ chuyển yêu cầu đến luồng xử lý tương ứng. Trong cả hai trường hợp, thiết bị chỉ thu nhận dữ liệu và hiển thị kết quả, còn quá trình suy luận của mô hình được thực hiện trên máy chủ.



Hình 5.2 Lưu đồ chức năng hệ thống

## ***Xây dựng ứng dụng nhận diện bọ cánh cứng***

Nếu dữ liệu đầu vào là ảnh máy chủ đưa ảnh vào mô hình để nhận diện sau đó bổ sung thông tin sinh học của loài từ cơ sở dữ liệu và trả kết quả về ứng dụng. Ứng dụng hiển thị ảnh đã được vẽ khung bao kèm tên loài, độ tin cậy và thông tin chi tiết của loài; nếu không phát hiện được đối tượng, ứng dụng thông báo không tìm thấy bọ trong ảnh.

Nếu dữ liệu đầu vào là video, máy chủ tách video thành các khung hình, lần lượt nhận diện trên từng khung, vẽ khung bao và nhãn loài rồi ghép lại thành video kết quả. Video này được trả về và phát trực tiếp trên ứng dụng, trong đó các cá thể bọ phát hiện được đều đã được đánh dấu sẵn; nếu không phát hiện được đối tượng, ứng dụng hiển thị thông báo tương ứng

### **5.4 Thiết kế cơ sở dữ liệu**

Do chức năng chính là tra cứu thông tin loài theo nhãn mà mô hình trả về, cơ sở dữ liệu được thiết kế quanh một bảng trung tâm duy nhất là bảng 5-1 bảng dữ liệu species, lưu toàn bộ thông tin chi tiết của chín loài bọ cánh cứng mà mô hình có khả năng nhận diện.

Bảng 5-1 Bảng dữ liệu species

| <b>Trường</b>     | <b>Kiểu dữ liệu</b> | <b>Mô tả</b>                                                                          |
|-------------------|---------------------|---------------------------------------------------------------------------------------|
| id                | INT                 | Khóa chính                                                                            |
| class_name        | VARCHAR(100)        | Nhãn lớp của loài, ràng buộc duy nhất và không rỗng; khớp với tên lớp mô hình xuất ra |
| ten_viet          | VARCHAR(200)        | Tên tiếng Việt của loài                                                               |
| ten_khoa_hoc      | VARCHAR(200)        | Tên khoa học của loài                                                                 |
| ho                | VARCHAR(200)        | Họ phân loại của loài                                                                 |
| kich_thuoc        | VARCHAR(100)        | Kích thước cơ thể                                                                     |
| mau_sac           | TEXT                | Mô tả màu sắc và hình thái                                                            |
| moi_truong        | TEXT                | Môi trường sống và cây ký chủ                                                         |
| dac_diem_sinh_hoc | TEXT                | Đặc điểm sinh học, vòng đời                                                           |
| gay_hai           | TEXT                | Cách thức và mức độ gây hại                                                           |
| phong_chong       | TEXT                | Các biện pháp phòng chống                                                             |
| hinh_anh_url      | VARCHAR(500)        | Đường dẫn đến ảnh đại diện của loài                                                   |
| hinh_anh_gay_hai  | VARCHAR(500)        | Đường dẫn đến ảnh minh họa tác hại                                                    |

### **5.5 Thiết kế API**

Các API được sử dụng trong hệ thống được thể hiện qua bảng 5-2.

## ***Xây dựng ứng dụng nhận diện bọ cánh cứng***

**Bảng 5-2 Tổng hợp các API sử dụng**

| <b>API</b>                | <b>Phương thức</b> | <b>Chức năng</b>           | <b>Dữ liệu trả về</b>                               |
|---------------------------|--------------------|----------------------------|-----------------------------------------------------|
| /api/predict              | POST               | Nhận diện loài từ ảnh      | Danh sách khung bao, độ tin cậy và thông tin loài   |
| /api/predict_video        | POST               | Nhận diện loài từ video    | Đường dẫn video kết quả và danh sách loài phát hiện |
| /api/species              | GET                | Lấy danh sách toàn bộ loài | Thông tin của tất cả các loài                       |
| /api/species/{class_name} | GET                | Lấy chi tiết một loài      | Thông tin đầy đủ của một loài                       |

Endpoint /api/predict xử lý nhận diện ảnh, nhận yêu cầu theo phương thức POST với dữ liệu dạng multipart/form-data gồm tệp ảnh và mã thiết bị. Khi nhận được ảnh, backend dùng thư viện Pillow để đọc ảnh rồi đưa trực tiếp vào mô hình nhận diện. Mô hình phát hiện các cá thể bọ và xuất ra tọa độ khung bao cùng độ tin cậy của từng đối tượng, kèm nhãn lớp tương ứng. Với mỗi nhãn lớp nhận được, backend truy vấn bảng species theo class\_name để lấy thông tin sinh học của loài như tên tiếng Việt, họ, mức độ nguy hiểm và biện pháp phòng chống. Toàn bộ kết quả gồm tọa độ khung bao và thông tin loài được đóng gói thành một đối tượng JSON và trả về cho ứng dụng.

Endpoint /api/predict\_video xử lý nhận diện video, cũng nhận yêu cầu theo phương thức POST với tệp video và mã thiết bị. Do video gồm nhiều khung hình nên luồng xử lý phức tạp hơn. Trước hết, tệp video được lưu tạm vào thư mục uploads\_tmp, sau đó backend dùng thư viện OpenCV để tách video thành từng khung hình. Để tối ưu hiệu năng, hệ thống áp dụng kỹ thuật bỏ khung với những khung bị bỏ qua, hệ thống dùng lại tọa độ nhận diện của khung ngay trước đó để khung bao vẫn bám theo đối tượng mượt mà trong khi tốc độ xử lý được cải thiện. Trên mỗi khung hình, khung bao và nhãn loài bằng tiếng Việt được vẽ thông qua sự kết hợp giữa OpenCV và Pillow. Sau khi vẽ xong, các khung hình được ghép lại và backend gọi công cụ FFmpeg qua tiến trình con để mã hóa video sang chuẩn nén H.264 bước bắt buộc để thư viện phát video của Flutter có thể đọc được. Tệp video kết quả được lưu vào thư mục processed\_videos, và backend trả về một đối tượng JSON chứa đường dẫn URL của video cùng danh sách các loài đã phát hiện để ứng dụng phát lại.

### ***Xây dựng ứng dụng nhận diện bọ cánh cứng***

---

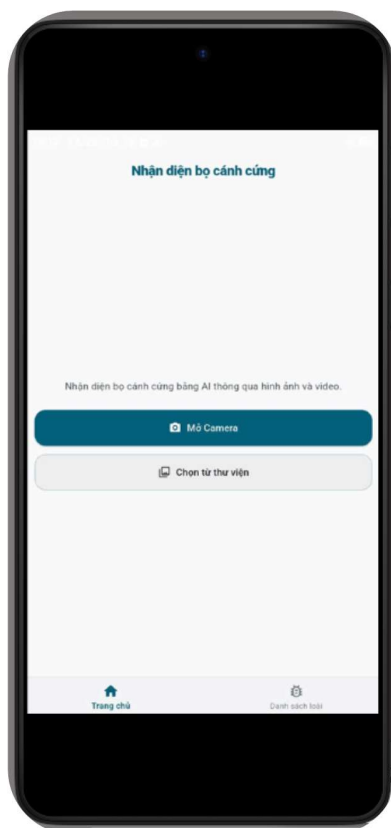
Endpoint `/api/species` trả về danh sách toàn bộ các loài trong cơ sở dữ liệu, phục vụ màn hình danh sách loài của ứng dụng. Endpoint `/api/species/{class_name}` trả về thông tin chi tiết của một loài cụ thể theo nhãn lớp truyền trên đường dẫn. Cả hai endpoint đều truy vấn dữ liệu thông qua ORM và trả kết quả về dưới dạng JSON.

## **Chương 6: KẾT QUẢ THỰC HIỆN**

Chương này giới thiệu về kết quả đã thực hiện được của đề tài bao gồm các hình ảnh giao diện, kết quả nhận diện trên ứng dụng thực tế đồng thời đánh giá hiệu năng ứng dụng.

### **6.1 Giao diện ứng dụng**

#### **6.1.1 Trang chủ ứng dụng**

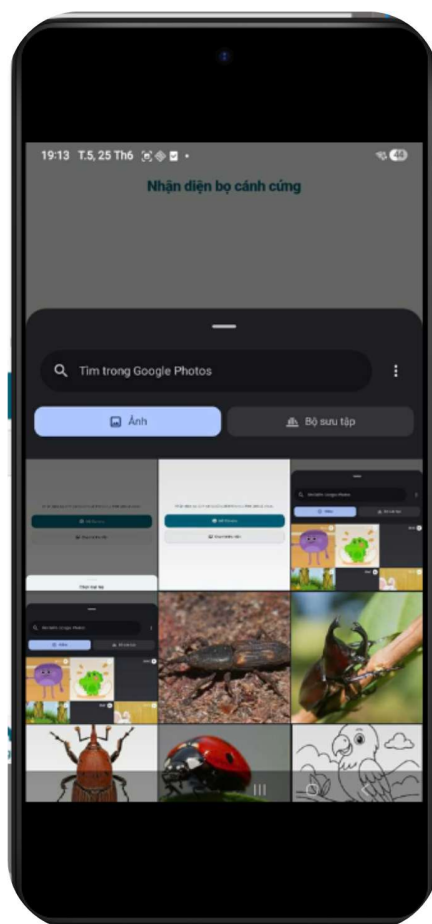


Hình 6.1 Giao diện trang chủ

Hình 6.1 thể hiện giao diện trang chủ của ứng dụng khi chạy trên thiết bị thực tế. Trung tâm màn hình bố trí hai nút thao tác chính: nút mở camera cho phép người dùng mở camera của ứng dụng và đưa ra lựa chọn chụp ảnh trực tiếp để nhận diện hoặc quay video để nhận diện, nút chọn từ thư viện cho phép chọn một ảnh hoặc video có sẵn trên thiết bị. Phía dưới cùng là thanh điều hướng gồm hai mục trang chủ và danh sách loài, giúp người dùng chuyển nhanh giữa màn hình nhận diện và màn hình tra cứu thông tin các loài.

### **6.1.2 Trang chọn dữ liệu tải lên**

Khi người dùng chọn nút chọn từ thư viện ở trang chủ, hiển thị popup để chọn giữa nhận diện hình ảnh và video sau đó ứng dụng mở giao diện thư viện của thiết bị để người dùng chọn dữ liệu cần nhận diện.

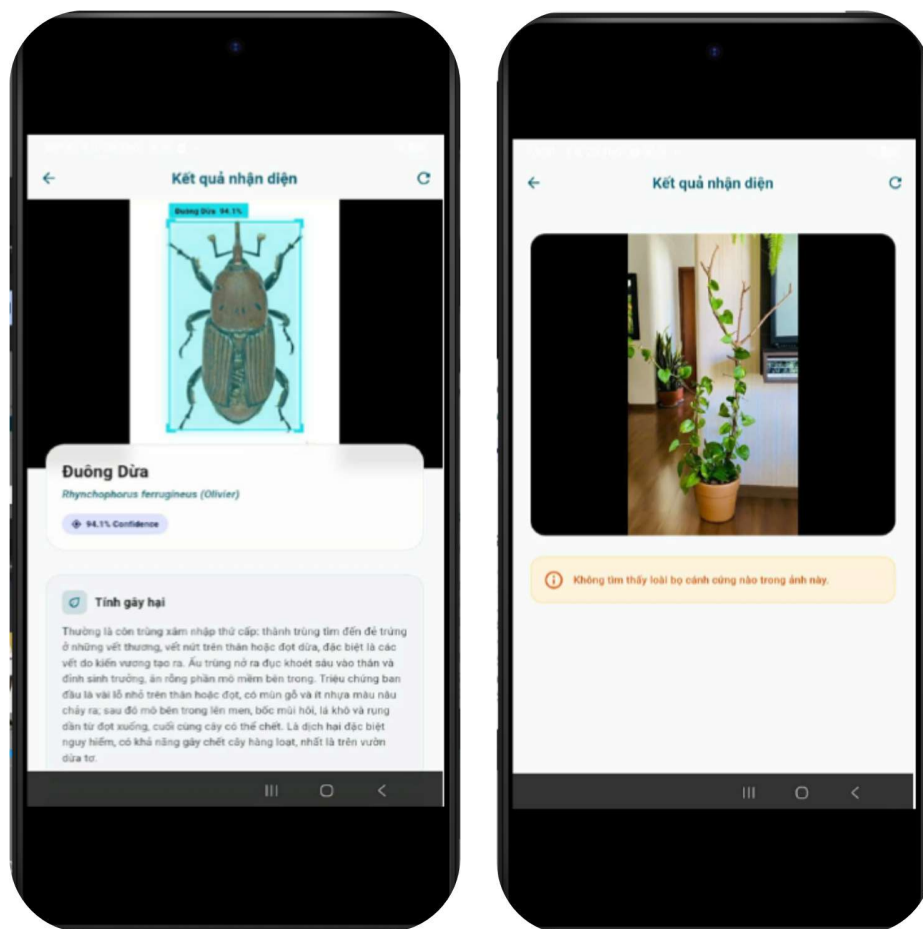


Hình 6.2 Trang chọn ảnh từ thư viện

Tại đây, người dùng có thể duyệt qua các tệp ảnh hoặc video có sẵn trên máy và chọn tệp mong muốn. Sau khi chọn, ứng dụng tiếp nhận tệp và gửi lên máy chủ để xử lý nhận diện. Trường hợp dữ liệu là ảnh, kết quả được hiển thị ở màn hình kết quả nhận diện; trường hợp dữ liệu là video, ứng dụng chuyển sang màn hình xử lý video.



### 6.1.3 Trang nhận diện ảnh

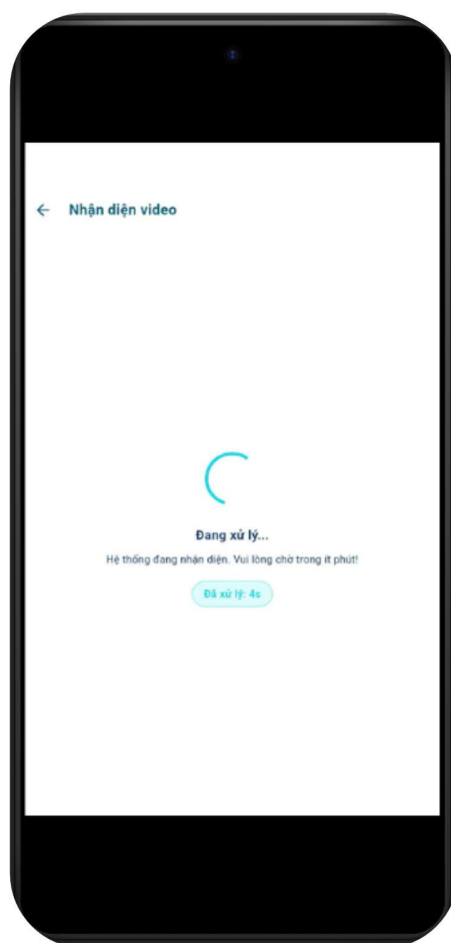


Hình 6.3 Trang kết quả nhận diện ảnh

Kết quả sau khi ứng dụng nhận diện thành công một ảnh được hiển thị tại trang kết quả nhận diện ảnh trên hình 6.3. Phía trên màn hình hiển thị ảnh đầu vào đã được vẽ khung bao quanh cá thể bọ phát hiện được, kèm nhãn tên loài và độ tin cậy ngay trên khung. Bên dưới ảnh là thẻ thông tin tóm tắt gồm tên tiếng Việt của loài, tên khoa học và độ tin cậy của kết quả nhận diện. Tiếp theo là các khối thông tin sinh học chi tiết của loài. Nếu ứng dụng không tìm thấy bọ trong ảnh ứng dụng hiển thị lại ảnh kèm dòng thông báo không tìm thấy.

### 6.1.4 Trang xử lý video

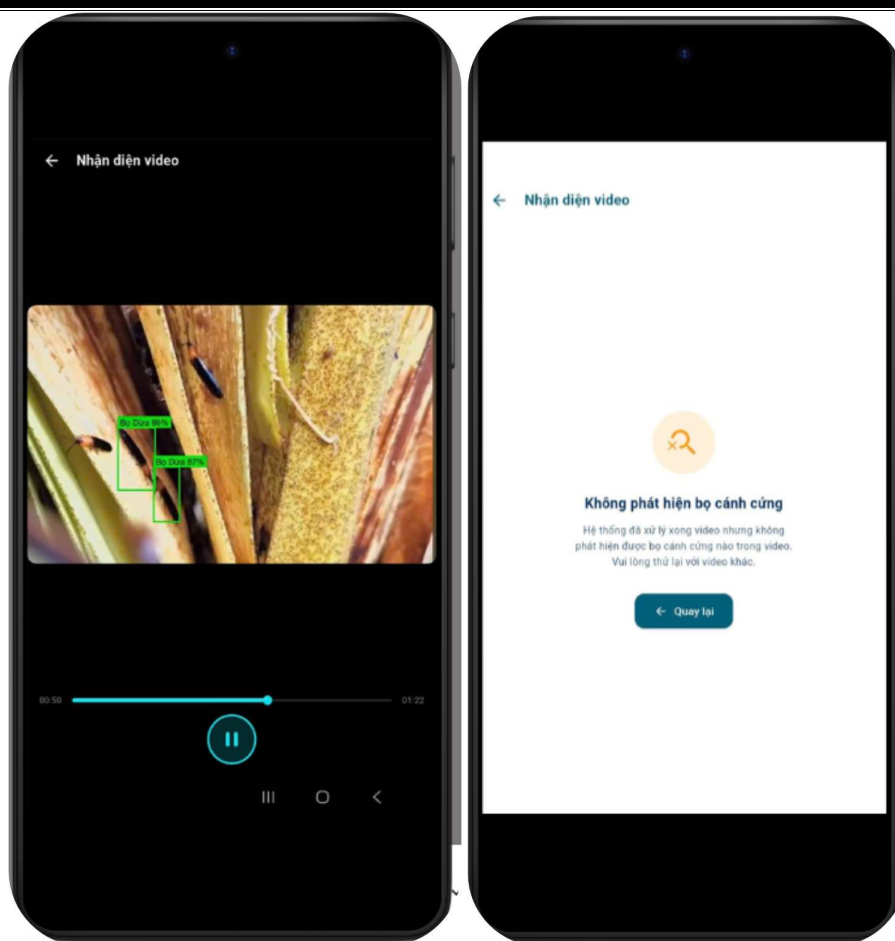
Hình 6.4 thể hiện màn hình chờ trong quá trình nhận diện video. Do việc xử lý video phải tách và nhận diện trên nhiều khung hình nên cần một khoảng thời gian nhất định; trong thời gian này, ứng dụng hiển thị màn hình chờ để thông báo cho người dùng.



Hình 6.4 Trang chờ xử lý

### **6.1.5 Trang kết quả nhận diện video**

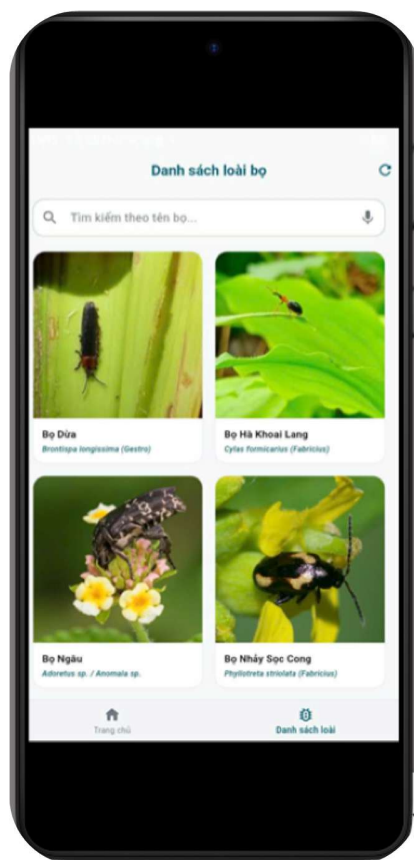
Video kết quả được phát trực tiếp trên màn hình, trong đó các cá thể bộ cánh cứng phát hiện được đều đã được vẽ khung bao kèm nhãn tên loài và độ tin cậy ngay trên từng khung. Phía dưới video là thanh tiến trình hiển thị thời lượng và vị trí đang phát, cùng nút tạm dừng hoặc tiếp tục phát, cho phép người dùng chủ động theo dõi kết quả. Các khung bao này được máy chủ vẽ sẵn lên video trong quá trình xử lý, do đó người dùng chỉ cần xem lại mà không cần thiết bị thực hiện thêm bất kỳ tính toán nào.



Hình 6.5 Kết quả nhận diện trên video

Trong trường hợp không phát hiện được loài bọ cánh cứng nào hiển thị màn hình không phát hiện bọ cánh cứng. Giữa màn hình là biểu tượng kính lúp kèm dòng tiêu đề không phát hiện bọ cánh cứng, bên dưới là đoạn thông báo giải thích rằng hệ thống đã xử lý xong video nhưng không tìm thấy bọ cánh cứng và đề nghị người dùng thử lại với video khác. Phía dưới có nút quay lại để người dùng trở về màn hình trước và thực hiện nhận diện với dữ liệu khác.

### 6.1.6 Trang danh sách loài



Hình 6.6 Trang danh sách loài

Trang hiển thị danh sách tất cả các loài ứng dụng có thể nhận diện được. Phía trên là thanh tìm kiếm cho phép người dùng tìm kiếm các loài bọ thông qua tên của chúng.

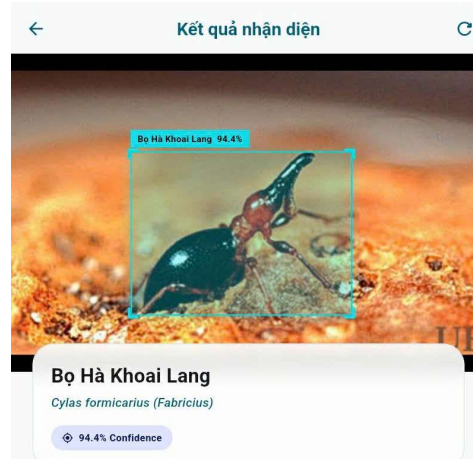
## 6.2 Kết quả nhận diện thực tế

### 6.2.1 Kết quả nhận diện trên ảnh

Ứng dụng vẽ khung bao quanh cá thể bọ phát hiện được, hiển thị nhãn tên loài và độ tin cậy ngay trên khung, đồng thời trình bày bên dưới các thông tin sinh học chi tiết của loài gồm tên tiếng Việt, tên khoa học, tính gây hại và biện pháp phòng chống. Kết quả kiểm thử hình 6.7 và hình 6.8 cho thấy ứng dụng nhận diện đúng loài với độ tin cậy cao, phù hợp với các chỉ số đánh giá mô hình đã đạt được ở chương trước. Đối với những ảnh không chứa bọ cánh cứng, hệ thống hiển thị thông báo không phát hiện được đối tượng.



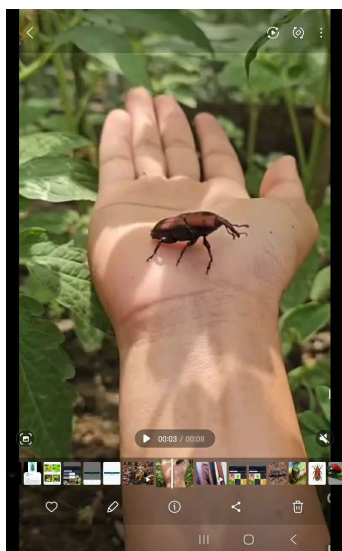
Hình 6.7 Ảnh gốc



Hình 6.8 Ảnh nhận diện

### 6.2.2 Kết quả nhận diện trên video

Kết quả nhận diện trên video được minh họa qua việc so sánh một khung hình trước và sau khi xử lý. Hình 5.9 là một khung hình trích từ video gốc, ghi lại cảnh một con bọ cánh cứng trên bàn tay người trong điều kiện quay thực tế ngoài vườn với phong nền phức tạp. Hình 6.10 là cùng khung hình đó sau khi được hệ thống xử lý: cá thể bọ đã được vẽ khung bao màu xanh kèm nhãn tên loài và độ tin cậy ngay trên khung, hệ thống nhận diện đúng loài với độ tin cậy cao.



Hình 6.9 Ảnh từ video gốc



Hình 6.10 Ảnh từ video nhận diện

### **6.3 Đánh giá hiệu năng ứng dụng**

Đối với nhận diện ảnh, ứng dụng trả về kết quả nhanh, trong khoảng vài giây kể từ khi gửi ảnh, đủ để người dùng sử dụng thuận tiện. Đối với nhận diện video, thời gian xử lý lâu hơn và phụ thuộc vào độ dài của video do phải xử lý nhiều khung hình; tuy nhiên nhờ kỹ thuật bỏ khung được áp dụng ở phía máy chủ, thời gian xử lý đã được rút ngắn đáng kể trong khi khung bao vẫn bám theo đối tượng tương đối mượt mà.

Hiệu năng của hệ thống còn phụ thuộc vào một số yếu tố. Thời gian xử lý video tỉ lệ với độ dài video, nên với các video dài thời gian chờ sẽ tăng lên. Ngoài ra, do hệ thống hoạt động theo mô hình client-server, tốc độ phản hồi còn chịu ảnh hưởng của chất lượng kết nối mạng giữa thiết bị và máy chủ. Cuối cùng, độ chính xác của kết quả nhận diện phụ thuộc vào chất lượng ảnh đầu vào như độ rõ nét, ánh sáng và góc chụp.

## **Chương 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN**

### **7.1 Kết luận**

Đề tài đã hoàn thành mục tiêu xây dựng một ứng dụng di động nhận diện bọ cánh cứng dựa trên học sâu, với một hệ thống hoàn chỉnh gồm mô hình nhận diện, ứng dụng di động, máy chủ và cơ sở dữ liệu.

#### **7.1.1 Ưu điểm**

Đề tài đã xây dựng được một hệ thống hoàn chỉnh theo kiến trúc client-server, trải dài từ khâu huấn luyện mô hình đến ứng dụng di động, máy chủ và cơ sở dữ liệu, vận hành thông suốt với nhau. Mô hình YOLO26n được huấn luyện trên chín loài bọ cánh cứng, đạt độ chính xác cao và nhận diện ổn định trong điều kiện thực tế. Ứng dụng hỗ trợ nhận diện trên cả ảnh và video, đồng thời cung cấp thông tin sinh thái chi tiết của từng loài, mang lại giá trị sử dụng thực tế cho người dùng. Nhờ đặt phần xử lý nặng ở máy chủ, ứng dụng di động được giữ nhẹ và chạy mượt ngay cả trên các thiết bị cấu hình không cao. Bên cạnh đó, giao diện được thiết kế đơn giản, trực quan và dễ thao tác đối với người dùng phổ thông.

#### **7.1.2 Nhược điểm**

Khi nhận diện trên ảnh, hệ thống thường chỉ phát hiện được một cá thể trong mỗi ảnh và đôi khi còn nhầm lẫn giữa đối tượng với phong nền, nhất là với những ảnh có nền phức tạp hoặc cá thể bọ có màu sắc gần giống nền. Tốc độ xử lý của hệ thống chưa thật sự nhanh, đặc biệt với đầu vào là video do phải xử lý nhiều khung hình. Số lượng loài nhận diện hiện còn giới hạn ở chín loài và nội dung thông tin cung cấp cho mỗi loài cũng còn hạn chế. Ngoài ra, hệ thống cần kết nối mạng đến máy chủ để hoạt động do toàn bộ phần xử lý nặng được đặt ở phía máy chủ.

## **7.2 Hướng phát triển**

Trên cơ sở những kết quả và hạn chế nêu trên, đề tài có thể được tiếp tục phát triển theo các hướng sau. Mở rộng số lượng loài nhận diện. Hiện tại hệ thống mới nhận diện được chín loài; trong tương lai có thể thu thập thêm dữ liệu và huấn luyện lại mô hình để bổ sung nhiều loài bọ cánh cứng khác, giúp ứng dụng bao quát tốt hơn nhu cầu thực tế của người dùng. Bổ sung chức năng xem lịch sử nhận diện. Hệ thống có thể lưu lại các kết quả mà người dùng đã nhận diện để họ xem lại khi cần, đồng thời phục vụ việc thống kê và theo dõi tình hình sâu hại theo thời gian. Cải thiện độ chính xác và khả năng phân biệt đối tượng với phong nền, đồng thời nâng cao khả năng phát hiện nhiều cá thể trong cùng một ảnh, bằng cách thu thập thêm dữ liệu đa dạng hơn về góc chụp, điều kiện ánh sáng và môi trường. Tối ưu tốc độ xử lý của hệ thống, đặc biệt là với đầu vào video, nhằm rút ngắn thời gian chờ và cải thiện trải nghiệm người dùng. Phát triển khả năng nhận diện trực tiếp theo thời gian thực qua camera, giúp người dùng nhận diện nhanh ngay tại hiện trường mà không cần chụp ảnh hay quay video trước. Nghiên cứu tích hợp mô hình chạy trực tiếp trên thiết bị di động để ứng dụng có thể hoạt động ngay cả khi không có kết nối mạng, đồng thời mở rộng và làm phong phú thêm nội dung thông tin cung cấp cho mỗi loài. Cuối cùng, hoàn thiện và triển khai chính thức ứng dụng lên các kho ứng dụng di động, kết hợp bổ sung các tính năng hỗ trợ người dùng như gợi ý biện pháp phòng trị phù hợp theo từng vùng và thu thập phản hồi của người dùng để liên tục cải thiện chất lượng mô hình.



## **PHỤ LỤC**

### **Xuất mô hình sang định dạng onnx**

Sau khi hoàn tất quá trình huấn luyện, mô hình YOLO26n (lưu ở tệp trọng số best.pt) được chuyển đổi sang định dạng ONNX (Open Neural Network Exchange) nhằm phục vụ triển khai trên nhiều nền tảng và môi trường chạy khác nhau, độc lập với framework huấn luyện ban đầu. Toàn bộ quá trình được thực thi trên nền tảng Google Colab.

Cài đặt thư viện Ultralytics

```
!pip install ultralytics
```

Nạp mô hình và xuất sang định dạng ONNX

```
from ultralytics import YOLO

model = YOLO("best.pt")

model.export(format="onnx", opset=11, dynamic=False)
```

Giải thích các tham số chính:

best.pt: nạp mô hình cùng bộ trọng số tốt nhất thu được sau quá trình huấn luyện.

format=onnx: chỉ định định dạng đầu ra là ONNX định dạng trung gian mở, cho phép sử dụng mô hình mà không phụ thuộc vào framework huấn luyện gốc.

opset=11: phiên bản tập toán tử (operator set) của ONNX, bảo đảm tính tương thích với các môi trường chạy (runtime) phổ biến.

dynamic=False: cố định kích thước đầu vào của mô hình, giúp đơn giản hóa và tối ưu quá trình suy luận khi triển khai.

Tệp kết quả best.onnx được sinh ra trong cùng thư mục với tệp trọng số gốc.

## DANH MỤC TÀI LIỆU THAM KHẢO

### Tài liệu tiếng Việt

[1] Nguyễn Thị Thu Cúc (2009), *Giáo trình Côn trùng nông nghiệp – Phần A: Côn trùng đại cương*, Trường Đại học Cần Thơ, Cần Thơ.

[2] Tạ Nguyễn Đức Dũng (2024), *Cải tiến và ứng dụng mô hình YOLOv9 để nhận dạng các bệnh trên cây lúa*, Đồ án tốt nghiệp, Đại học Bách khoa Hà Nội, Hà Nội.

[3] Nguyễn Văn Huỳnh, Lê Thị Sen (2003), *Giáo trình Côn trùng nông nghiệp – Phần B: Côn trùng gây hại cây trồng chính ở Đồng bằng sông Cửu Long*, Trường Đại học Cần Thơ, Cần Thơ.

[4] Nguyễn Đức Khiêm (2005), *Giáo trình Côn trùng nông nghiệp*, Nxb Nông nghiệp, Hà Nội.

[5] Chang A Náng (2020), *Nghiên cứu tính đa dạng các loài côn trùng bộ Cánh cứng (Coleoptera) và đề xuất biện pháp quản lý chúng ở xã Tủa Thành, huyện Tủa Chùa, tỉnh Điện Biên*, Khóa luận tốt nghiệp, Trường Đại học Lâm nghiệp Việt Nam, Hà Nội.

[6] Trung tâm Thông tin và Thống kê Khoa học và Công nghệ (2024), “Ứng dụng AI trong quản lý bệnh hại cây trồng”, [<https://thongke.cesti.gov.vn/>] (truy cập ngày 25 tháng 5 năm 2026).

### Tài liệu tiếng nước ngoài

[7] Fielding, R. T. (2000), *Architectural Styles and the Design of Network-based Software Architectures*, Luận án Tiến sĩ, University of California, Irvine.

[8] GeeksforGeeks (2025), “Introduction to Deep Learning”, [<https://www.geeksforgeeks.org/deep-learning/introduction-deep-learning/>] (truy cập ngày 02 tháng 5 năm 2026).

[9] GeeksforGeeks (2025), “What is Object Detection in Computer Vision?”, [<https://www.geeksforgeeks.org/computer-vision/what-is-object-detection-in-computer-vision/>] (truy cập ngày 05 tháng 5 năm 2026).

- [10] Goodfellow, I., Bengio, Y. and Courville, A. (2016), *Deep Learning*, MIT Press, Cambridge, MA.
- [11] Google (2018), *Flutter - Build apps for any screen*, [Available: <https://flutter.dev/>] (truy cập ngày 13 tháng 5 năm 2026).
- [12] Hansen, O. L. P., Svenning, J.-C., Olsen, K., Dupont, S., Garner, B. H., Iosifidis, A., Price, B. W. and Høye, T. T. (2020), “Species-level image classification with convolutional neural network enables insect identification from habitus images”, *Ecology and Evolution*.
- [13] Hidayatullah, P. and Tubagus, R. (2026), “YOLO26: A Comprehensive Architecture Overview and Key Improvements”.
- [14] Mitchell, T. M. (1997), *Machine Learning*, McGraw-Hill, New York.
- [15] Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016), “You Only Look Once: Unified, Real-Time Object Detection”, *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788.
- [16] Roboflow (2026), “Import from Roboflow Universe”, *Roboflow Docs*, [<https://docs.roboflow.com/datasets/adding-data/roboflow-universe>] (truy cập ngày 08 tháng 5 năm 2026).
- [17] Roboflow (2026), “Use Roboflow Annotate”, *Roboflow Docs*, [<https://docs.roboflow.com/annotate/use-roboflow-annotate>] (truy cập ngày 08 tháng 5 năm 2026).
- [18] Russell, S. and Norvig, P. (2021), *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, Hoboken, NJ.
- [19] Szeliski, R. (2022), *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, Cham, Switzerland.
- [20] Ultralytics (2024), *Ultralytics YOLO*, [<https://github.com/ultralytics/ultralytics>] (truy cập ngày 02 tháng 5 năm 2026).

[21] Ultralytics (2024), *Ultralytics YOLO Documentation*,  
[<https://docs.ultralytics.com/>] (truy cập ngày 02 tháng 5 năm 2026).

[22] Ultralytics (2026), *Ultralytics YOLO26*,  
[<https://docs.ultralytics.com/models/yolo26/>] (truy cập ngày 02 tháng 5 năm 2026).